

SAINS DATA DENGAN PYTHON

Modul praktikum ini dirancang untuk mendukung proses pembelajaran mata kuliah Sains Data pada jenjang Sarjana (S1)

Prasyarat

Praktikan telah mengenal:
Algoritma dan Pemrograman -
Matematika Diskrit / Aljabar Linier dasar -
Statistika Dasar -

RINALDI HAMZAH
5220411342

MODUL PRAKTIKUM

SAINS DATA DENGAN PYTHON

Program Studi : Informatika (S1)
Peruntukan : Mahasiswa Sarjana (S1)
Tools : Google Colab, Python 3, Pandas, NumPy, Matplotlib, Scikit-Learn
Penyusun : Dosen Informatika
Semester : 7

Capaian Pembelajaran Mata Kuliah (CPMK)

Setelah mengikuti seluruh rangkaian praktikum, mahasiswa diharapkan mampu:

- **M1:** Menggunakan bahasa pemrograman Python untuk penyelesaian masalah sains data
- **M2:** Menggunakan *Pandas* untuk data analisis dan *NumPy* untuk data numerik
- **M3:** Menggunakan *Matplotlib* untuk visualisasi data dan *statistical plot*
- **M4:** Menggunakan *Scikit-Learn* untuk penyelesaian kasus *machine learning*
- **M5:** Mengimplementasikan model sains data dengan Python terhadap suatu kasus

Penilaian Praktikum

- **30%:** Kehadiran dan partisipasi praktikum
- **40%:** Laporan setiap praktikum (Notebook + dokumentasi analisis)
- **30%:** Proyek akhir integratif

Catatan:

Setiap praktikum dilengkapi dengan *notebook template* yang dapat disalin (*duplicate*) melalui Google Colab. Mahasiswa wajib menjalankan kode, menganalisis output, serta mendokumentasikan hasil analisis secara sistematis.

PENDAHULUAN

Modul praktikum ini disusun sebagai panduan pelaksanaan pembelajaran praktis pada mata kuliah **Sains Data**, yang dirancang untuk mendukung pencapaian **Capaian Pembelajaran Mata Kuliah (CPMK)**. Setiap sesi praktikum mengintegrasikan konsep teoritis dengan implementasi langsung menggunakan bahasa pemrograman Python melalui lingkungan **Google Colab**.

Melalui modul ini, mahasiswa diharapkan mampu memahami dan mengimplementasikan tahapan utama dalam proses sains data, mulai dari pengolahan dan analisis data, visualisasi, hingga pembangunan dan evaluasi model *machine learning* secara sistematis dan terstruktur.

DAFTAR ISI

MODUL PRAKTIKUM SAINS DATA DENGAN PYTHON	ii
PENDAHULUAN	iii
DAFTAR ISI	iv
Praktikum 1 Pengantar Python untuk Sains Data (CPMK M1)	1
Tujuan Praktikum:	1
Materi Pokok:	1
Contoh Kode:.....	1
Praktikum 2 Analisis Data dengan Pandas dan NumPy (CPMK M2)	37
Tujuan Praktikum:	37
Materi Pokok:	37
Contoh Kode:.....	Error! Bookmark not defined.
Praktikum 3 Visualisasi Data dengan Matplotlib (CPMK M3)	48
Tujuan Praktikum:	48
Materi Pokok:	48
Contoh Kode:.....	Error! Bookmark not defined.
Praktikum 4 Machine Learning dengan Scikit-Learn (CPMK M4)	58
Tujuan Praktikum:	58
Materi Pokok:	58
Contoh Kode:.....	Error! Bookmark not defined.
Praktikum 5 Proyek Integratif Sains Data (CPMK M5)	65
Tujuan Praktikum:	65
Materi Pokok:	65
Contoh Studi Kasus:.....	65
Alur Pengerjaan:.....	65
Tugas Akhir:	65

Praktikum 1

Pengantar Python untuk Sains Data (CPMK M1)

Tujuan Praktikum:

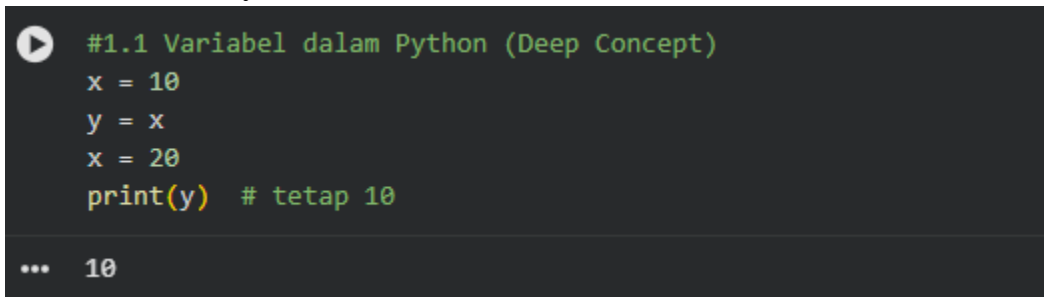
Mahasiswa mampu menguasai sintaks dasar Python serta menggunakan struktur data dan fungsi dasar untuk menyelesaikan permasalahan sederhana dalam sains data.

Materi Pokok:

1. Variabel, tipe data, dan struktur data (list, dictionary, tuple, set)
2. Struktur kontrol: percabangan dan perulangan
3. Fungsi dan modularisasi program
4. Manipulasi string dan *file I/O*
5. Pengenalan Google Colab dan alur kerja *notebook*

1.1 Variabel, tipe data dan struktur data (list, dictionary,tuple,set)

1. Variabel dalam Python



```
#1.1 Variabel dalam Python (Deep Concept)
x = 10
y = x
x = 20
print(y) # tetap 10

... 10
```

Keterangan:

- ☐ Variabel tidak menyimpan nilai, tetapi mereferensikan objek di memori
- ☐ Python adalah dynamically typed language
- ☐ Tipe data ditentukan saat runtime

```

a = [1, 2, 3]
b = a
a.append(4)
print(b) # ikut berubah

... [1, 2, 3, 4]

```

Keterangan:

Penambahan nilai 4 ke dalam list dan akan di panggil dengan perintah print(b).

2. Tipe Data

```

#1.2 tipe data
x = 1
type(x)

... int

```

Keterangan:

Intenger merupakan angka mutlak seperti 1,2,3,4 dan seterusnya tanpa adanya koma.

```

a = "Rinaldi"
type(a)

... str

```

Keterangan:

String merupakan hurup dasar tanpa adanya campuran dengan angka atau karakter yang lain.

```

a = 10
b = 3.5
c = a + b
print(type(c)) # float

... <class 'float'>

```

Keterangan:

Float merupakan gabungan dari angka, hurup dan karakter yang lain.

```

# Boolean adalah tipe data yang digunakan untuk
# menyimpan nilai kebenaran,
# yaitu hanya dua kemungkinan:
# True (benar)
# False (salah)
x = True
type(x)

... bool

```

Keterangan:

Boolean mempunyai 2 karakteristik yaitu benar dan salah atau menggunakan dua opsi.

```

# None adalah tipe data khusus yang digunakan untuk
# menyatakan bahwa tidak ada nilai, kosong, atau
# nilai belum ditentukan.
e = None
type(e)

... NoneType

```

Keterangan:

Bentuk data None atau nilai kosong.

3. Struktur Data List

```

#1.3 Struktur Data List
my_list = [78, 89, 23, 45, 56]
type(my_list)

... list

print(my_list)

[78, 89, 23, 45, 56]

```

Keterangan:

Struktur List biasa menggunakan angka atau number.

```

mixed_list = [23, 3.14, "apple", True, None]
type(mixed_list)

list

print(mixed_list)

[23, 3.14, 'apple', True, None]

```

Keterangan:

Memasukkan semua tipe data dalam satu list sehingga menjadi mix-list atau list campuran.

```

#function in list
#min()--return the minimum number of the list
#Metode min() digunakan untuk mengembalikan nilai
#minimum dari sebuah list (atau iterable lainnya).
my_list = [98, 34, 56, 45, 12]
min(my_list)

... 12

#max()--return maximum number of the list
#Metode max() digunakan untuk mengembalikan nilai
#maksimum dari sebuah list (atau iterable lainnya).
max(my_list)

98

#sum()--return addition of the all item of the list
#Fungsi sum() digunakan untuk menjumlahkan semua elemen
#dalam list (atau iterable).
sum(my_list)

245

```

Keterangan:

Menggunakan perintah min, max dan sum di dalam sebuah list.

```

#sorted()--this function arrange list increasing
#or decreasing order
#Fungsi sorted() digunakan untuk mengurutkan elemen
#list (atau iterable) tanpa mengubah list asli.
list1 = [9, 7, 12, 4, 8, 3]
sorted(list1, reverse = True)

... [12, 9, 8, 7, 4, 3]

```

Keterangan:

Sorted digunakan untuk mengurutkan list dari yang terkecil ke terbesar atau sebaliknya.


```
#reverse()-- this method reverse the list
#Metode .reverse() digunakan untuk membalik
#urutan elemen dalam list secara langsung (in-place).
my_list = [90, 87, 56, 23]
my_list.reverse()
print(my_list)

[23, 56, 87, 90]
```

Keterangan:

Membalik urutan elemen data dalam list secara langsung (in-place).

```
#append()--this method add item end of the list
#and take item as argument
#Metode .append() digunakan untuk menambahkan satu
#elemen baru di akhir list.
#Elemen yang ditambahkan diberikan sebagai argumen.
my_list = [90, 87, 65, 34]
print(my_list)
my_list.append(100)
print(my_list)

... [90, 87, 65, 34]
[90, 87, 65, 34, 100]
```

Keterangan:

Menambahkan angka atau nilai ke dalam suatu list dengan menggunakan perintah append.

```
#extend()--this method merge a list into another list
#Metode .extend() digunakan untuk menambahkan semua
#elemen dari satu list ke list lain.
#Berbeda dengan .append(), .extend() menambahkan elemen
#satu per satu, bukan sebagai satu item tunggal.
list1 = [90, 78, 65]
list2 = [56, 89, 45]
list1.extend(list2)
print(list1)

... [90, 78, 65, 56, 89, 45]
```

Keterangan:

Menggabungkan 2 list menjadi 1 list dengan menggunakan fungsi extend.

```

▶ #pop()--this method remove item from the list,
  #take index as argument
  #Metode .pop() digunakan untuk menghapus elemen dari
  #list berdasarkan
  #indeks dan mengembalikan elemen yang dihapus.
  my_list = [78, 56, 34, 12]
  my_list.pop()

... 12

```

Keterangan:

Metode .pop() digunakan untuk menghapus elemen dari list berdasarkan indeks dan mengembalikan elemen yang dihapus.

```

#remove()--this method remove item of the list,
#take item as a argument
#Metode .remove() digunakan untuk menghapus elemen
#tertentu dari list berdasarkan nilainya, bukan indeks.
list1 = [90, 87, 65, 23]
list1.remove(87)
list1

[90, 65, 23]

```

Keterangan:

Metode .remove() digunakan untuk menghapus elemen tertentu dari list berdasarkan nilainya, bukan indeks.

4. Tuple (Lebih dari Sekadar List Tetap)

```

#1.4 Tuple
# Tuple dasar
data_tuple = (10, 20, 30)

print(data_tuple)
print(type(data_tuple))

(10, 20, 30)
<class 'tuple'>

```

Keterangan:

Tuple dasar dengan menggunakan data number atau numerik.

```

def statistik(data):
    minimum = min(data)
    maksimum = max(data)
    rata_rata = sum(data) / len(data)
    return minimum, maksimum, rata_rata

nilai = [80, 90, 75, 85]
hasil = statistik(nilai)

print("Min:", hasil[0])
print("Max:", hasil[1])
print("Mean:", hasil[2])

... Min: 75
    Max: 90
    Mean: 82.5

min_val, max_val, mean_val = statistik(nilai)
print(min_val, max_val, mean_val)

75 90 82.5

```

Keterangan:

Menggunakan banyak nilai statistik secara bersamaan dan menggunakan perintah aritmatik.

```

#Iterasi Tuple
data = (5, 10, 15)

for nilai in data:
    print(nilai)

5
10
15

```

Keterangan:

Menggunakan iterasi tuple pada data di dalam tuple.

```

#Perbandingan list dan tuple
list_data = [1, 2, 3]
tuple_data = (1, 2, 3)
print(type(list_data))
print(type(tuple_data))

... <class 'list'>
    <class 'tuple'>

```

5. Set

```
#1.5 Set
data_set = {1, 2, 3, 3, 3, 4, 4, 5}
print(data_set) # duplikat otomatis dihapus

{1, 2, 3, 4, 5}
```

Set adalah struktur data:

- Tidak berurutan
- Tidak memiliki indeks
- Tidak menyimpan duplikat
- Bersifat mutable

```
#Menambah & Menghapus Elemen
angka = {1, 2, 3, 5, 7, 8}
angka.add(4)
angka.remove(2)
print(angka)

{1, 3, 4, 5, 7, 8}
```

Keterangan:

Menambah dan mengurangi data di dalam lis dengan menggunakan perintah add dan remove.

```
#Operasi Set
A = {1, 2, 3}
B = {3, 4, 5}
print("Union:", A | B)
print("Intersection:", A & B) #angka yang sama
print("Difference:", A - B)
print("Symmetric Difference:", A ^ B)

Union: {1, 2, 3, 4, 5}
Intersection: {3}
Difference: {1, 2}
Symmetric Difference: {1, 2, 4, 5}
```

Keterangan:

Operasi yang digunakan pada data yang berbentuk set biasanya secara umum.

```
#Set untuk Menghilangkan Duplikat Data
data = [10, 20, 20, 30, 30, 40]
uniq = set(data)
print(uniq)

{40, 10, 20, 30}

#Set Comprehension
data = [1, 2, 3, 4, 5, 6]
genap = {x for x in data if x % 2 == 0}
print(genap)

... {2, 4, 6}

#Pengecekan Keanggotaan
kategori = {"A", "B", "C"}
print("A" in kategori)
print("D" in kategori)

True
False
```

Keterangan:

Set untuk menghilangkan data uniq, data comprehension dan pengecekan keanggotaan dalam kategori.

```
#Contoh kasus sederhana
data1 = ["Jakarta", "Bandung", "Surabaya"]
data2 = ["Bandung", "Medan", "Surabaya"]
kota_sama = set(data1) & set(data2)
kota_uniq = set(data1) | set(data2)
print("Kota Sama:", kota_sama)
print("Kota Unik:", kota_uniq)

Kota Sama: {'Surabaya', 'Bandung'}
Kota Unik: {'Jakarta', 'Bandung', 'Surabaya', 'Medan'}
```

Keterangan:

Contoh kasus sederhana menggunakan set.

6. Dictionary

```

#1.6 Dictionary
#dictionary
#Dictionary adalah tipe data di Python yang digunakan
#untuk menyimpan data dalam bentuk pasangan key: value.
my_dict = {"Name": "Rinaldi",
           "College": "Universitas Teknologi Yogyakarta",
           "Major": "Informatika"}
type(my_dict)

... dict

print(my_dict)
print(my_dict["Name"])
print(my_dict["College"])
print(my_dict["Major"])

{'Name': 'Rinaldi', 'College': 'Universitas Teknologi Yogyakarta', 'Major': 'Informatika'}
Rinaldi
Universitas Teknologi Yogyakarta
Informatika

```

Dictionary adalah struktur data:

- Menyimpan pasangan key : value
- Key harus unik & immutable
- Mutable (value bisa diubah)

```

#method in dictionary
#keys()--this method find the keys of dictionary
#Metode .keys() digunakan untuk mengambil semua
#key dari dictionary sebagai view object.
list(my_dict.keys())

['Name', 'College', 'Major']

#values()--this method find value of the dictionary
#Metode .values() digunakan untuk mengambil semua
#value dari dictionary sebagai view object
list(my_dict.values())

... ['Rinaldi', 'Universitas Teknologi Yogyakarta', 'Informatika']

#items()--this method find items of the dictionary
#Metode .items() digunakan untuk mengambil semua
#pasangan key: value dari dictionary sebagai view object.
list(my_dict.items())

[('Name', 'Rinaldi'),
 ('College', 'Universitas Teknologi Yogyakarta'),
 ('Major', 'Informatika')]

```

Keterangan:

Menggunakan keys(), values() dan items().

```

#get()--this method find value in the dictionary
#and key take as argument
#Metode .get() digunakan untuk mengambil nilai
#dari dictionary berdasarkan key.
#Jika key tidak ada, tidak menimbulkan error,
#melainkan mengembalikan None
#(atau nilai default jika diberikan).
my_dict.get("Major")

... 'Informatika'

#update()--this method update the dictionary
#and take dictionary as argument
#Metode .update() digunakan untuk menambahkan
#atau memperbarui key-value dari dictionary
#menggunakan dictionary lain sebagai argumen.
my_dict.update({"Age":20})
print(my_dict)

... rsitas Teknologi Yogyakarta', 'Major': 'Informatika', 'Age': 20}

```

Keterangan:

Menggunakan metode get() dan update().

7. Nested Dictionary

```

#1.7 Nested Dictionary
data_mahasiswa = {
    "A001": {
        "nama": "Ani",
        "umur": 20,
        "nilai": 85
    },
    "A002": {
        "nama": "Budi",
        "umur": 21,
        "nilai": 90
    }
}
print(data_mahasiswa["A001"]["nama"])
print(data_mahasiswa["A002"]["nilai"])

Ani
90

```

Keterangan:

Nested Dictionary adalah dictionary yang di dalamnya berisi dictionary lain sebagai value.

```
#Iterasi Nested Dictionary
for nim, info in data_mahasiswa.items():
    print("NIM:", nim)
    for key, value in info.items():
        print(f" {key}: {value}")

... NIM: A001
    nama: Ani
    umur: 20
    nilai: 85
NIM: A002
    nama: Budi
    umur: 21
    nilai: 90
```

Keterangan:

Iterasi nested dictionary secara manual dengan memanggil value keys.

```
#Menambah & Mengubah Data
data_mahasiswa["A003"] = {
    "nama": "Cici",
    "umur": 19,
    "nilai": 78
}

data_mahasiswa["A001"]["nilai"] = 88
```

Keterangan:

Menambah dan mengurangi data secara manual.

```
#Menghitung Rata-rata Nilai
total = 0
for mhs in data_mahasiswa.values():
    total += mhs["nilai"]

rata_rata = total / len(data_mahasiswa)
print("Rata-rata nilai:", rata_rata)

... Rata-rata nilai: 85.33333333333333
```

Keterangan:

Menghitung rata-rata nilai yang berada di dalam dictionary.


```
#Filter data
lulus = {}
for nim, info in data_mahasiswa.items():
    if info["nilai"] >= 80:
        lulus[nim] = info
print(lulus)

... {'A001': {'nama': 'Ani', 'umur': 20, 'nilai': 88}, 'A002': {'na
```

Keterangan:

Melakukan filter data nilai dengan parameter besar sama 80 akan di tampilkan menjadi hasil print(lulu).

```
#Handling Missing Key
nilai = data_mahasiswa.get("A004", {}).get("nilai", "Tidak ada")
print(nilai)

... Tidak ada data

#Konversi ke Struktur Data
#Ke list of dictionary
records = []
for nim, info in data_mahasiswa.items():
    record = {"nim": nim}
    record.update(info)
    records.append(record)
print(records)

... [{'nim': 'A001', 'nama': 'Ani', 'umur': 20, 'nilai': 88}, {'nim
```

Keterangan:

Melakukan handling missing value dengan fungsi get, dan mengkonversi data menjadi list of dictionary.

8. Type Hinting

```
#1.8 Type Hinting
x: int = 10
y: float = 3.5
nama: str = "Python"
status: bool = True
```

Type Hinting adalah cara memberi petunjuk tipe data pada:

- variabel

- parameter fungsi
- nilai balik (return)
Python tidak memaksa tipe (bukan statically typed), tapi membantu:
- keterbacaan kode
- debugging
- kolaborasi tim
- analisis data skala besar

```
#Type Hinting pada Fungsi
def rata_rata(data: list[float]) -> float:
    return sum(data) / len(data)
```

```
#Menggunakan Modul typing
from typing import List, Dict, Tuple
def statistik(data: List[int]) -> Tuple[int, int, float]:
    return min(data), max(data), sum(data) / len(data)
```

```
#Type Hinting Dictionary & Nested Dictionary
from typing import Dict
Mahasiswa = Dict[str, int | str]
data_mahasiswa: Dict[str, Mahasiswa] = {
    "A001": {"nama": "Ani", "nilai": 85},
    "A002": {"nama": "Budi", "nilai": 90}
}
```

Keterangan:

Import menggunakan library List, Dict, dan Tuple.

```
#Optional & Union
from typing import Optional, Union
def cari_nilai(nim: str) -> Optional[int]:
    return data_mahasiswa.get(nim, {}).get("nilai")
def konversi(x: Union[int, float]) -> float:
    return float(x)
```

```
#Type Hinting untuk List of Tuple
from typing import List, Tuple
Dataset = List[Tuple[str, int, float]]
data: Dataset = [
    ("A001", 20, 85.5),
    ("A002", 21, 90.0)
]
```

Keterangan:

Menggunakan union dan List of Tuple.

```
#Type Hinting untuk Return Banyak Nilai
from typing import Tuple
def min_max(data: list[int]) -> Tuple[int, int]:
    return min(data), max(data)
```

Keterangan:

Return banyak nilai menggunakan tuple.

1.2 Struktur kontrol: percabangan dan perulangan

1. Percabangan (if, elif, else)

```
#Percabangan (if, elif, else)
#Percabangan Dasar
nilai = 85

if nilai >= 90:
    grade = "A"
elif nilai >= 80:
    grade = "B"
elif nilai >= 70:
    grade = "C"
else:
    grade = "D"
print("Grade:", grade)

... Grade: B
```

Keterangan:

Percabangan atau conditional menggunakan fungsi if, elif dan else.

```
#Logical Operator (Advance)
umur = 20
izin = True

if umur >= 18 and izin:
    print("Boleh masuk")

... Boleh masuk
```

Keterangan:

Operator logik menggunakan percabangan if.

```

▶ #Nested If (Data Filtering)
nilai = 90
kehadiran = 80

if nilai >= 75:
    if kehadiran >= 75:
        print("Lulus")
    else:
        print("Tidak lulus (absensi)")

... Lulus

```

Keterangan:

Nested if berfungsi untuk melakukan filter data dengan parameter tertentu.

```

▶ #Ternary Operator (Pythonic)
status = "Lulus" if nilai >= 75 else "Tidak Lulus"
print(status)

... Lulus

```

Keterangan:

Ternary operator dengan perulangan sederhana if dan else.

2. Perulangan (for, while)

```

▶ #Perulangan for
data = [10, 20, 30]
for x in data:
    print(x)

... 10
    20
    30

```

Keterangan:

Perulangan menggunakan for in data.

```

#range() (Iterasi Numerik)
for i in range(1, 6):
    print(i)

1
2
3
4
5

```

Keterangan:

Perulangan menggunakan for in range dengan memanggil parameter (i).

```
#Perulangan while
x = 0
while x < 5:
    print(x)
    x += 1
```

0
1
2
3
4

Keterangan:

Perulangan menggunakan while dengan parameter tertentu.

3. Kontrol Perulangan (break, continue)

```
#break
for x in range(10):
    if x == 5:
        break
    print(x)
```

... 0
1
2
3
4

```
#continue
for x in range(5):
    if x == 2:
        continue
    print(x)
```

0
1
3
4

Keterangan:

Menggunakan break dan continue perbedaanya adalah jika kita menggunakan break maka loop akan berhenti secara total dan continue merupakan berhenti di salah satu angka

atau baris saja contoh `x==2`: jadi loop akan melewati angka 2 saja dan lanjut sampai habis seluruh data.

4. Perulangan pada Struktur Data

```
#Loop List
nilai = [80, 90, 70]
for i in nilai:
    print(i)

... 80
     90
     70

#Loop Dictionary
mahasiswa = {"Ani": 85, "Budi": 90}
for nama, nilai in mahasiswa.items():
    print(nama, nilai)

Ani 85
Budi 90
```

Keterangan:

Perulangan di dalam struktur data yaitu list dan dictionary.

```
#Loop Nested Dictionary
data = {
    "A001": {"nilai": 85},
    "A002": {"nilai": 90}
}
for nim, info in data.items():
    print(nim, info["nilai"])

... A001 85
     A002 90
```

Keterangan:

Perulangan di dalam dictionary atau sering kita sebut data json.

5. `enumerate()` dan `zip()` (Advance & Pythonic)

```

#enumerate()
data = ["A", "B", "C"]
for i, val in enumerate(data):
    print(i, val)

... 0 A
    1 B
    2 C

#zip()
nama = ["Ani", "Budi"]
nilai = [85, 90]
for n, v in zip(nama, nilai):
    print(n, v)

Ani 85
Budi 90

```

Keterangan:

Enumerate() merupakan pengurutan data dari nilai 0 dan Zip() menggabungkan kedua data yang berada dalam struktur data.

6. Loop + Kondisi (Data Filtering)

```

data = [70, 80, 90, 60]

lulus = []

for n in data:
    if n >= 75:
        lulus.append(n)

print(lulus)

[80, 90]

```

Keterangan:

Menggunakan loop dan kondisi terhadap data filtering dengan parameter tertentu.

7. List Comprehension

```
data = [70, 80, 90, 60]

lulus = [n for n in data if n >= 75]
print(lulus)

[80, 90]
```

Keterangan:

Melakukan pemanggilan data langsung tanpa menggunakan list baru untuk tempat hasil data.

8. Error Handling dalam Struktur Kontrol

```
data = ["10", "20", "x"]

for d in data:
    try:
        print(int(d))
    except ValueError:
        print("Data tidak valid:", d)

10
20
Data tidak valid: x
```

Keterangan:

Cek data yang merupakan intenger di dalam list jika tidak ada makan print error.

1.3 Fungsi dan modularisasi program

1. Konsep Dasar

```
#konsep dasar fungsi
def great():
    print("Halo, Data Science")

#Fungsi dengan Parameter & Return
def tambah(a, b):
    return a + b

hasil = tambah(3, 5)
print(hasil)
```

... 8

Fungsi digunakan untuk:

- menghindari pengulangan kode
- meningkatkan keterbacaan
- memudahkan debugging
- membangun pipeline data science

```
#Fungsi dengan Default Parameter
def pajak(harga, pajak=0.1):
    return harga + (harga * pajak)
```

```
print(pajak(10000))
print(pajak(10000, 0.2))
```

```
11000.0
12000.0
```

```
#Fungsi Return Banyak Nilai (Tuple)
def min_max(data):
    return min(data), max(data), sum(data) / len(data)
```

```
nilai = [80, 90, 70, 75, 98, 100]
nilai_min, nilai_max, nilai_rata = min_max(nilai)
print(nilai_min, nilai_max, nilai_rata)
```

```
70 100 85.5
```

Keterangan:

Fungsi ini menggunakan nilai tuple untuk menghirung min, max dan mean.

```
#Fungsi dengan Type Hinting (Best Practice)
from typing import List, Tuple
def statistik(data: List[int]) -> Tuple[int, int, float]:
    return min(data), max(data), sum(data) / len(data)
print(statistik([1, 2, 3, 4, 5]))
```

```
(1, 5, 3.0)
```

```
#Fungsi sebagai Argumen (Functional Programming)
def pangkat(x):
    return x ** 2
data = [1, 2, 3, 4, 5]
hasil = list(map(pangkat, data))
print(hasil)
```

```
[1, 4, 9, 16, 25]
```

```
#Lambda Function (Ringkas & Pythonic)
data = [1, 2, 3, 4, 5]
hasil = list(map(lambda x: x ** 2, data))
print(hasil)
```

```
[1, 4, 9, 16, 25]
```

Keterangan:

Menggunakan import list dan tuple untuk memanggil angka statistik yang sudah ada.

2. Modularisasi Program

```
project/
| — main.py
| — statistik.py

#statistik.py
def mean(data):
    return sum(data) / len(data)

def median(data):
    data = sorted(data)
    n = len(data)
    mid = n // 2
    return (data[mid] if n % 2 != 0 else (data[mid - 1] + data

#main.py
from statistik import mean, median

data = [10, 20, 30, 40]
print(mean(data))
print(median(data))
```

Keterangan:

Memanggil dua fungsi yaitu def mean dan def median dengan menggunakan satu directory baru yaitu main.py.

```
def main():
    print("Program dijalankan")

if __name__ == "__main__":
    main()

... Program dijalankan
```

Keterangan:

Isinya adalah kode utama program, Tidak otomatis dijalankan saat didefinisikan, Jalankan main() HANYA jika file ini dijalankan langsung, bukan di-import.

1.4 Manipulasi string dan file Input/Output

1. Manipulasi String

```
#Konsep string
#String adalah data teks yang sangat sering digunakan
#dalam sains data
teks = "Belajar Data Science dengan Python"
print(teks.lower())      # huruf kecil
print(teks.upper())      # huruf besar
print(teks.title())      # format judul
print(teks.replace("Python", "Rinaldi"))

... belajar data science dengan python
BELAJAR DATA SCIENCE DENGAN PYTHON
Belajar Data Science Dengan Python
Belajar Data Science dengan Rinaldi
```

Keterangan:

String adalah **data teks** yang sangat sering digunakan dalam sains data (contoh: nama, kategori, komentar, dokumen).

```
#Menghapus Spasi & Karakter Tidak Perlu
data = "  Data, Science!  "
data = data.strip()
data = data.lower()
data = data.replace(",","").replace("!", "")
print(data)

data science

#Memecah String (split)
kalimat = "python untuk sains data"
kata = kalimat.split()
print(kata)

['python', 'untuk', 'sains', 'data']

#Menggabungkan String (join)
kata = ["data", "science", "python"]
kalimat = " ".join(kata)
print(kalimat)

data science python
```

Keterangan:

Melakukan penghapusan karakter yang tidak perlu memecah string menjadi split dan menggabungkan string.

```
#Iterasi String
kata = "science"
for huruf in kata:
    print(huruf)
```

```
... s
    c
    i
    e
    n
    c
    e
```

Keterangan:

Iterasi kata pada tipe data string.

2. File Input / Output (File I/O)

```
#Menulis File
with open("hasil.txt", "w") as file:
    file.write("Belajar Python untuk Data Science")
```

```
#Membaca File Teks
with open("hasil.txt", "r", encoding="utf-8") as file:
    isi = file.read()
    print(isi)
```

```
Belajar Python untuk Data Science
```

```
#Menambahkan Isi File (Append)
with open("hasil.txt", "a") as file:
    file.write("\nBaris tambahan")
```

```
#Membaca File per Baris
with open("hasil.txt", "r") as file:
    for baris in file:
        print(baris.strip())
```

```
... Belajar Python untuk Data Science
    Baris tambahan
```

Keterangan:

Langkah-langkah yang dilakukan merupakan menulis, membaca, memabahkan dan membaca file di setiap baris.

1.5 Pengenalan Google Colab dan alur kerja notebook

1. Pengertian Google Colab

Google Colaboratory (Google Colab) adalah lingkungan pemrograman berbasis web yang memungkinkan pengguna menulis dan menjalankan kode Python langsung melalui browser tanpa perlu melakukan instalasi perangkat lunak apa pun di komputer.

Google Colab berbasis Jupyter Notebook, sehingga pengguna dapat menggabungkan:

- 1) Kode program Python
- 2) Teks penjelasan (Markdown)
- 3) Output hasil eksekusi (teks, tabel, grafik)

Software ini pada dasarnya serupa dengan Jupyter Notebook gratis berbentuk *cloud* yang dijalankan menggunakan *browser*, seperti Mozilla Firefox dan Google Chrome. euntungan terbesar dari Google Colaboratory adalah bahwa ia memiliki kumpulan *built-in-library machine learning* paling populer yang dapat dimuat dengan mudah dalam *notebook*.

2. Keunggulan Google Colab untuk Praktikum

Google Colab sendiri merupakan aplikasi yang diciptakan khusus operasi *machine learning* dan *deep learning*. Aplikasi ini memiliki banyak fitur menarik yang membedakannya dari *software* pengkodean lainnya.

Manfaat yang ditawarkan Google Colab, Berikut penjelasannya menurut Towards Data Science.

- 1) *built-in-library machine learning* yang lengkap
- 2) berbasis *cloud*, sehingga tidak memakan *space* dalam memori komputer
- 3) data dalam Google Colaboratory dapat diakses dan diedit dengan mudah
- 4) mempermudah proses kolaborasi antar tim
- 5) memiliki fitur GPU dan TPU yang dapat dimanfaatkan secara gratis

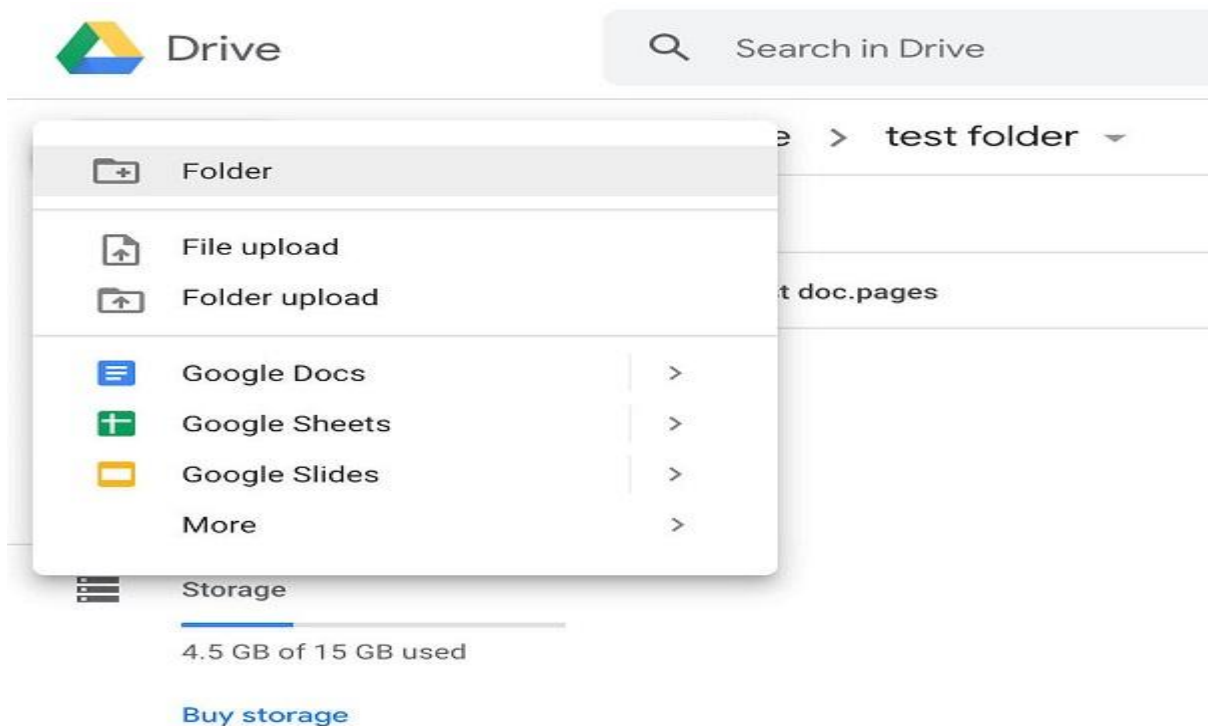
Google Colab adalah *executable document* yang bisa dimanfaatkan guna menyimpan, menulis, dan membagikan program yang sudah ditulis dari Google Drive. Google Colab sering digunakan karena memungkinkan penggunaanya untuk menjalankan kode Python tanpa perlu melakukan proses instalasi.

3. Cara Menggunakan Google Colab



Pasalnya, Google Colaboratory dapat dimanfaatkan untuk melakukan tugas tertentu dalam paradigma berorientasi sel, serupa dengan Jupyter Notebook. Tak hanya itu, *software* tersebut juga dapat digunakan untuk membuat berbagai tipe sel dan menciptakan *notebook*, seperti halnya fitur-fitur Jupyter Notebook.

1) Membuat folder di Google Drive

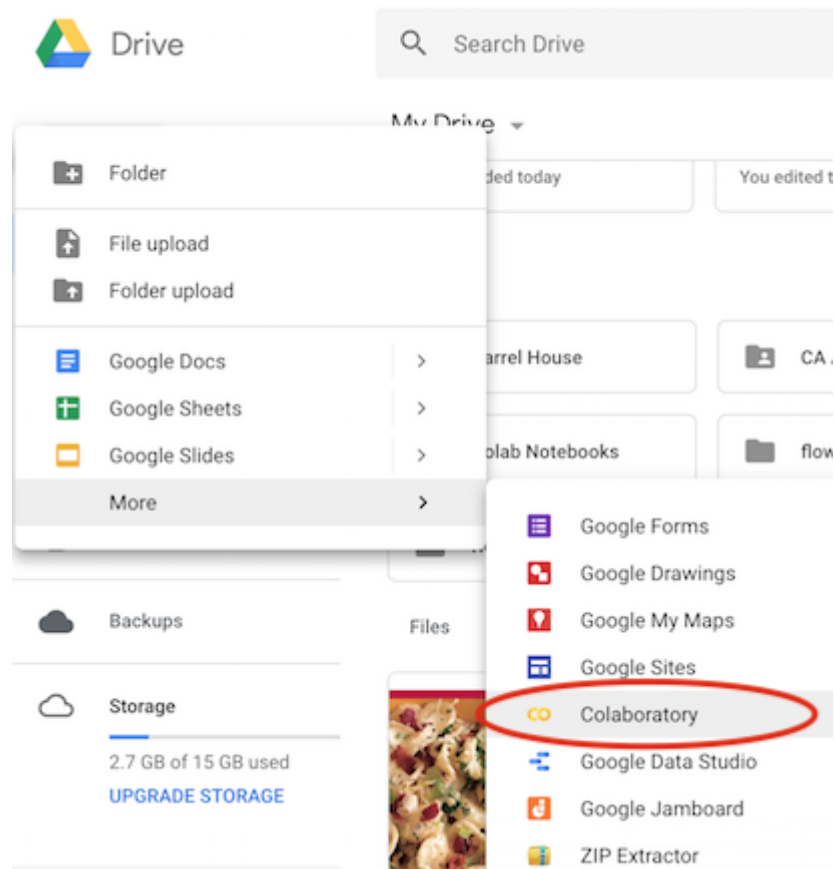


Pertama-tama, untuk menggunakan Google Colab, kamu harus memiliki akun Google lalu kemudian akses fitur Colaboratory. Jika tidak memiliki akun Google, sebagian besar dari fitur Colaboratory yang perlu kamu akses tidak akan berfungsi.

Lalu, dikarenakan Google Colaboratory bekerja dalam Google Drive, kamu harus menentukan *folder* yang akan digunakan.

Beri nama *folder* tersebut menggunakan nama baru atau dengan judul *default* yang sudah disediakan Google Colaboratory.

2) Membuat Notebook



Google Colab sudah terintegrasi dengan folder di Drive, berarti kamu sudah siap untuk menggunakannya.

Namun, kamu harus buat *file* Notebook baru terlebih dahulu dengan cara klik kanan di dalam *folder* yang baru saja kita buat, pilih More dan lalu klik opsi Colaboratory. Setelah itulah baru fitur-fitur yang tersedia dalam Google Colaboratory dapat kamu manfaatkan.

4. Struktur Notebook Google Colab

1) Buka Broser

google.colab

colab.google
https://colab.google

Google Colab

Colab is a hosted Jupyter Notebook service that requires no setup to use and provides free access to computing resources, including GPUs and TPUs. [Read more](#)

colab.google
https://colab.google › notebooks

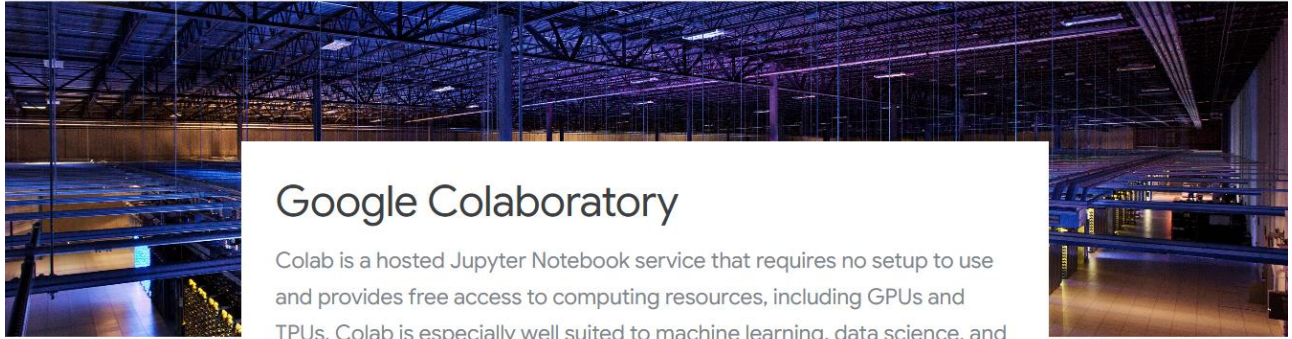
Curated Notebooks

The goal of this Colab notebook is to **highlight some benefits of using Google BigQuery and Colab together** to perform some common data science tasks. Online ... [Read more](#)

2) Akses: <https://colab.research.google.com>

Google colab Blog Release Notes Notebooks Resources

[Open Colab](#) [New Notebook](#) [Sign Up](#)



Google Colaboratory

Colab is a hosted Jupyter Notebook service that requires no setup to use and provides free access to computing resources, including GPUs and TPUs. Colab is especially well suited to machine learning, data science, and education.

[Open Colab](#) [New Notebook](#)

3) Login

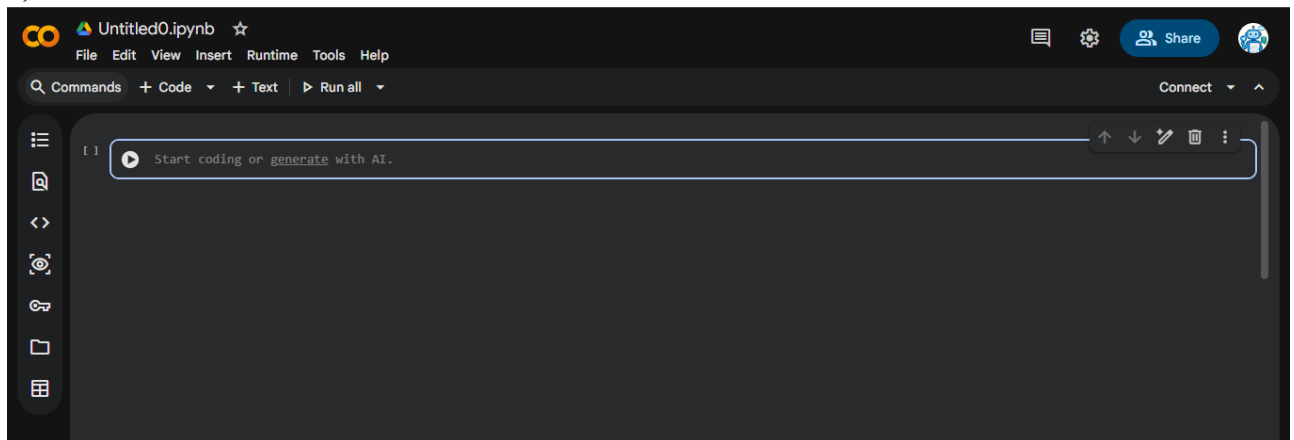
colab

Choose the Colab plan that's right for you

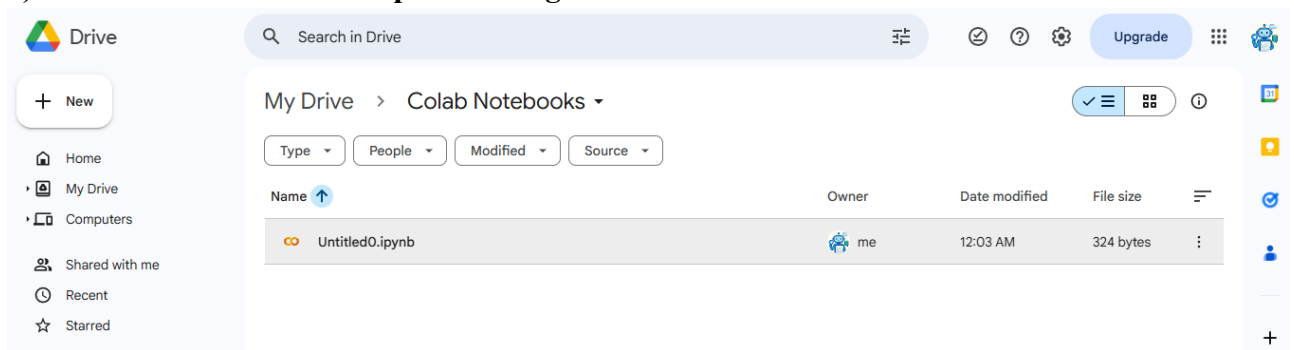
Whether you're a student, a hobbyist, or a ML researcher, Colab has you covered
Colab is always free of charge to use, but as your computing needs grow there are paid options to meet them.
[Restrictions apply. learn more here](#)

Pay As You Go	Recommended Colab Pro	Colab Pro+	Colab Enterprise
US\$9.99 for 100 Compute Units US\$49.99 for 500 Compute Units You currently have 0 compute units. Compute units expire after 90 days. Purchase more as you need them.	US\$9.99 per month ✓ 100 compute units per month. Compute units expire after 90 days. Purchase more as you need them. ✓ Faster GPUs	US\$49.99 per month All of the benefits of Pro, plus: ✓ An additional 500 compute units per month, totaling 600 compute units per month. Compute units expire after 90 days.	Pay for what you use ✓ Integrated. Tightly integrated with Google Cloud services like BigQuery and Vertex AI. ✓ Enterprise notebook storage

4) Pilih New Notebook



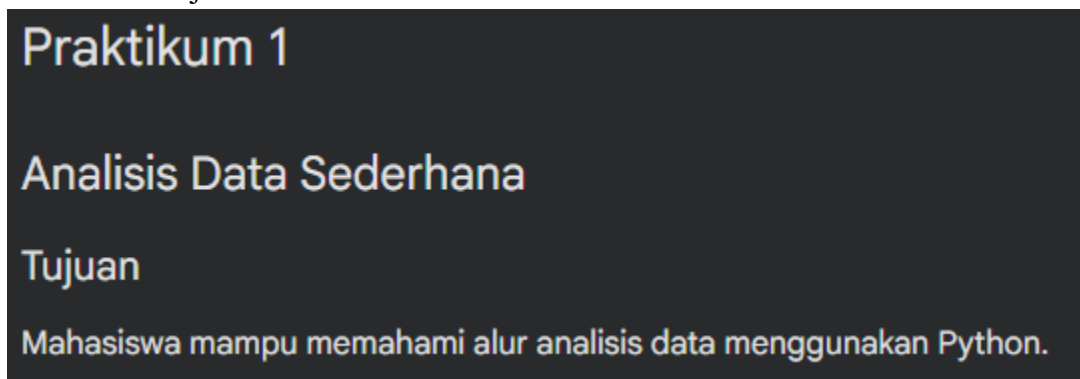
5) Notebook otomatis tersimpan di Google Drive



5. Alur Kerja (Workflow) Notebook Sains Data

Workflow notebook sains data adalah urutan langkah kerja terstruktur dalam sebuah notebook (Google Colab/Jupyter) yang digunakan untuk melakukan analisis data secara sistematis, mulai dari penjelasan masalah hingga penarikan kesimpulan.

□ Judul & Tujuan



□ Import Library



Keterangan:

Memanggil pustaka Python yang dibutuhkan dan menyiapkan lingkungan kerja.

☐ Input Data

```
#Input Data
data = [10, 20, 30, 40, 50]
```

Keterangan:

Memasukkan data yang akan dianalisis dan data dapat berasal dari input manual atau file.

☐ Proses Data

```
#Proses Data
rata_rata = sum(data) / len(data)
nilai_maks = max(data)
nilai_min = min(data)
```

☐ Hasil & Output

```
#Hasil dan Output
print("Rata-rata:", rata_rata)
print("Nilai maksimum:", nilai_maks)
print("Nilai minimum:", nilai_min)

... Rata-rata: 30.0
    Nilai maksimum: 50
    Nilai minimum: 10
```

☐ Kesimpulan

Kesimpulan

Rata-rata data adalah 30, dengan nilai maksimum 50 dan minimum 10.

Alur kerja notebook sains data merupakan fondasi utama dalam praktikum pengantar sains data. Dengan workflow yang terstruktur, mahasiswa dapat memahami tidak hanya cara menulis kode, tetapi juga cara berpikir sebagai analis data.

6. Peran Notebook dalam Sains Data

1. Pengertian Notebook dalam Sains Data

Notebook (seperti Google Colab atau Jupyter Notebook) adalah dokumen interaktif yang menggabungkan kode program, teks penjelasan, dan output hasil eksekusi dalam satu lingkungan kerja. Dalam sains data, notebook berperan sebagai: media kerja utama untuk melakukan, mendokumentasikan, dan menyajikan proses analisis data

2. Notebook sebagai Media Eksplorasi Data

Notebook memungkinkan analisis data untuk:

- 1) mencoba berbagai pendekatan analisis
- 2) menjalankan kode secara bertahap
- 3) langsung melihat hasil dari setiap langkah

Proses ini dikenal sebagai Exploratory Data Analysis (EDA).

Contoh aktivitas:

- 1) melihat ringkasan data
- 2) mencoba perhitungan statistik
- 3) mengamati pola awal data

3. Notebook sebagai Media Dokumentasi Analisis

Notebook tidak hanya berisi kode, tetapi juga:

- 1) penjelasan langkah-langkah analisis
- 2) alasan pemilihan metode
- 3) interpretasi hasil

Dengan menggunakan Markdown, notebook berfungsi sebagai: laporan analisis data yang terdokumentasi dengan baik

4. Notebook sebagai Alat Pembelajaran

Dalam konteks pendidikan, notebook:

- 1) membantu mahasiswa memahami alur berpikir analisis data
- 2) memungkinkan pembelajaran bertahap (step-by-step)
- 3) menghubungkan teori dan praktik langsung

Mahasiswa dapat:

- 1) membaca penjelasan
- 2) menjalankan kode
- 3) mengamati output secara langsung dalam satu dokumen.

5. Notebook sebagai Media Eksperimen

Sains data bersifat eksperimental. Notebook mendukung:

- 1) percobaan berbagai parameter
- 2) modifikasi kode tanpa merusak keseluruhan program
- 3) evaluasi hasil secara cepat

Hal ini menjadikan notebook ideal untuk:

- 1) uji coba model
- 2) perbandingan metode

- 3) eksplorasi ide baru

6. Notebook sebagai Media Kolaborasi

Notebook dapat:

- 1) dibagikan secara online
- 2) dikerjakan bersama
- 3) digunakan untuk diskusi dan evaluasi

Dalam Google Colab:

- 1) dosen dapat memberikan komentar
- 2) mahasiswa dapat mengumpulkan tugas melalui satu file notebook

7. Notebook sebagai Jembatan ke Sistem Produksi

Meskipun notebook digunakan untuk eksplorasi, hasil analisis di dalamnya dapat:

- 1) dijadikan dasar pembuatan script Python
- 2) dikembangkan menjadi aplikasi
- 3) diintegrasikan ke sistem produksi

Dengan demikian, notebook berperan sebagai:

tahap awal sebelum implementasi sistem sains data yang lebih besar

Tugas Praktikum:

1. Buat fungsi Python untuk menghitung **mean, median, dan modus** dari sebuah list numerik.

Penjelasan Singkat

- 1) Mean (rata-rata): jumlah seluruh data dibagi banyaknya data
- 2) Median: nilai tengah setelah data diurutkan
- 3) Modus: nilai yang paling sering muncul

Kode Program

```
#kode program
def hitung_mean(data: list[float]) -> float:
    return sum(data) / len(data)

def hitung_median(data: list[float]) -> float:
    data_urut = sorted(data)
    n = len(data_urut)
    tengah = n // 2

    if n % 2 == 0:
        return (data_urut[tengah - 1] + data_urut[tengah]) / 2
    else:
        return data_urut[tengah]

def hitung_modus(data: list[float]) -> float | list[float]:
    frekuensi = {}
    for nilai in data:
        frekuensi[nilai] = frekuensi.get(nilai, 0) + 1
    max_frekuensi = max(frekuensi.values())
    modus = [k for k, v in frekuensi.items() if v == max_frekuensi]
    if len(modus) == 1:
        return modus[0]
    return modus

data = [10, 20, 20, 30, 40, 50, 60, 70, 80, 90, 100]

print("Mean:", hitung_mean(data))
print("Median:", hitung_median(data))
print("Modus:", hitung_modus(data))

... Mean: 51.81818181818182
    Median: 50
    Modus: 20
```

Penjelasan Cara Kerja

Mean

Menggunakan `sum()` dan `len()` untuk menghitung rata-rata.

Median

Data diurutkan terlebih dahulu.

- Jika jumlah data ganjil → ambil nilai tengah
- Jika genap → rata-rata dua nilai tengah

Modus

Menggunakan dictionary untuk menghitung frekuensi setiap nilai.

Jika terdapat lebih dari satu nilai dengan frekuensi tertinggi, fungsi mengembalikan list modus.

2. Baca sebuah file teks dan hitung frekuensi kemunculan setiap kata unik.

Langkah Penyelesaian

1. Membuka dan membaca file teks
2. Mengubah teks menjadi huruf kecil (agar konsisten)
3. Menghilangkan tanda baca
4. Memecah teks menjadi kata-kata
5. Menghitung frekuensi setiap kata menggunakan dictionary

Kode Program

```
#Menulis File
with open("data.txt", "w") as file:
    file.write("Belajar Python untuk Data Science, Python adalah bahasa pemrograman,
```

```
#kode program
def hitung_frekuensi_kata(nama_file: str) -> dict[str, int]:
    frekuensi = {}

    with open(nama_file, "r", encoding="utf-8") as file:
        teks = file.read().lower()

    for karakter in ".!?:;":
        teks = teks.replace(karakter, "")

    kata_kata = teks.split()

    for kata in kata_kata:
        frekuensi[kata] = frekuensi.get(kata, 0) + 1

    return frekuensi
```

```
▶ hasil = hitung_frekuensi_kata("data.txt")

for kata, jumlah in hasil.items():
    print(kata, ":", jumlah)

... belajar : 1
   python : 4
   untuk : 2
   data : 3
   science : 3
   adalah : 1
   bahasa : 1
   pemrograman : 1
   digunakan : 1
   menggunakan : 1
```

Penjelasan Cara Kerja

- 1) open()
Membuka file teks untuk dibaca.
- 2) lower()
Menyamakan huruf agar Python dan python dianggap sama.
- 3) replace()
Menghapus tanda baca agar tidak dihitung sebagai kata berbeda.
- 4) split()
Memecah teks menjadi list kata.
- 5) dictionary
Menyimpan kata sebagai key dan jumlah kemunculan sebagai value.

Praktikum 2

Analisis Data dengan Pandas dan NumPy (CPMK M2)

Tujuan Praktikum:

Mahasiswa mampu melakukan manipulasi dan analisis data tabular menggunakan Pandas serta komputasi numerik menggunakan NumPy.

Materi Pokok:

1. Struktur data Pandas: *Series* dan *DataFrame*
2. *Filtering, grouping, dan aggregation*
3. Operasi array NumPy: *slicing, broadcasting, dan universal functions*
4. Penanganan data hilang (*missing values*)

2.1 Struktur data Pandas: Series dan DataFrame

```
Library

#Library
import pandas as pd
import numpy as np
```

Keterangan:

Import library untuk operasi numerik dan data proseding

```
df = pd.read_excel('/content/drive/MyDrive/Semester7 2026/Data')
#semua bentuk karakteristik data bisa di import seperti excel,
df.head() #melihat 5 baris data pertama
```

```
...   pizza_id  order_id  pizza_name_id  quantity  order_date  order_
```

1.0	1.0	hawaiian_m	1.0	1/1/2015	11:3
2.0	2.0	classic_dlx_m	1.0	1/1/2015	11:3

Keterangan:

Import data menggunakan excel.

```
df.info() #Melihat karakteristik data
```

```
... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 48620 entries, 0 to 48619
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   pizza_id               48620 non-null  float64
1   order_id               48620 non-null  float64
2   pizza_name_id          48620 non-null  object
3   quantity               48620 non-null  float64
4   order_date              48620 non-null  object
5   order_time              48620 non-null  object
6   unit_price              48620 non-null  float64
7   total_price             48620 non-null  float64
8   pizza_size              48620 non-null  object
9   pizza_category          48620 non-null  object
10  pizza_ingredients       48620 non-null  object
11  pizza_name              48620 non-null  object
dtypes: float64(5), object(7)
memory usage: 4.5+ MB
```

Keterangan:

Info merupakan melihat tipe data dan karakteristik data yang sudah di import.

```
df.describe() #mendeskripsikan bentuk data
```

	pizza_id	order_id	quantity	unit_price
count	48620.000000	48620.000000	48620.000000	48620.000000
mean	24310.500000	10701.479761	1.019622	16.494132
std	14035.529381	6180.119770	0.143077	3.621789
min	1.000000	1.000000	1.000000	9.750000
25%	12155.750000	5337.000000	1.000000	12.750000
50%	24310.500000	10682.500000	1.000000	16.500000
75%	36465.250000	16100.000000	1.000000	20.250000
max	48620.000000	21350.000000	4.000000	35.950000

Keterangan:

Mendeskripsikan data secara keseluruhan.

```
df.isnull().sum() #menghitung jumlah data yang hilang
#di dalam data ini ternyata tidak ada missing value
```

	0
pizza_id	0
order_id	0
pizza_name_id	0
quantity	0
order_date	0
order_time	0
unit_price	0
total_price	0
pizza_size	0
pizza_category	0
pizza_ingredients	0
pizza_name	0

Keterangan:

Melakukan cek missing values di seluruh data.

2.2 Filtering, Grouping, dan Aggregation

Cleaning

```
#menghapus angka nul
df['pizza_id'] = df['pizza_id'].astype(int)
df['order_id'] = df['order_id'].astype(int)
df['quantity'] = df['quantity'].astype(int)

df['unit_price'] = df['unit_price'].astype(int)
df['total_price'] = df['total_price'].astype(int)
```

Keterangan:

Mengganti data yang berbentuk float menjadi intenger, agar datanya bersih.

```
df['order_date'] = pd.to_datetime(
    df['order_date'],
    format='mixed',
    dayfirst=True
)

df['order_time'] = pd.to_datetime(
    df['order_time'],
    format='mixed'
).dt.time
```

Keterangan:

Mengganti format data tanggal dan waktu menjadi lebih bagus formatnya.

```
df['pizza_size'] = df['pizza_size'].str.lower().str.strip()
df['pizza_category'] = df['pizza_category'].str.lower().str.st
df['pizza_name'] = df['pizza_name'].str.lower().str.strip()
df['pizza_ingredients'] = (
    df['pizza_ingredients']
    .str.lower()
    .str.replace(['^a-z,'], '', regex=True)
    .str.strip()
)
```

Keterangan:

Menyamakan data yang berbentuk dokumen kata menjadi sama semua.

```
df.info()
df.head()
```

```
... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 48620 entries, 0 to 48619
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   pizza_id              48620 non-null  int64
1   order_id              48620 non-null  int64
2   pizza_name_id         48620 non-null  object
3   quantity              48620 non-null  int64
4   order_date            48620 non-null  datetime64[ns]
5   order_time            48620 non-null  object
6   unit_price            48620 non-null  int64
7   total_price           48620 non-null  int64
8   pizza_size            48620 non-null  object
9   pizza_category        48620 non-null  object
10  pizza_ingredients     48620 non-null  object
11  pizza_name            48620 non-null  object
12  price_check           48620 non-null  int64
dtypes: datetime64[ns](1), int64(6), object(6)
memory usage: 4.8+ MB
```

Keterangan:

Datanya sudah dapat berganti menjadi lebih bersih lagi.

```
GROUPING (Pengelompokan Data)
```

```
#Total revenue per kategori pizza
revenue_category = df.groupby('pizza_category')['total_price']
revenue_category
```

```
...                total_price
pizza_category
chicken          187867
classic          216159
supreme          202078
veggie           189716

dtype: int64
```

Keterangan:

Melakukan grubping data antara pizza kategory dam total price.

```
#Total jumlah pizza terjual per ukuran
quantity_size = df.groupby('pizza_size')['quantity'].sum()
quantity_size
```

quantity	
pizza_size	
l	18956
m	15635
s	14403
xl	552
xxl	28

dtype: int64

Keterangan:

Groubby berdasarkan ukuran pizza dan quantity dari penjualan pizza.

```
AGGREGATION (Agregasi Statistik)

#Agregasi lengkap per kategori
summary_category = df.groupby('pizza_category').agg(
    total_revenue=('total_price', 'sum'),
    total_quantity=('quantity', 'sum'),
    avg_unit_price=('unit_price', 'mean'),
    total_orders=('order_id', 'nunique')
)
summary_category
```

	total_revenue	total_quantity	avg_unit_price
pizza_category			
chicken	187867	11050	16.959408
classic	216159	14888	14.527128
supreme	202078	11987	16.840282
veggie	189716	11649	16.263342

Keterangan:

Agregasi statistik di beberapa kategori data.

2.3 Operasi array NumPy:slicing, broadcasting dan universal functions

SLICING (Memotong Data)

```
# ambil kolom numerik penting
pizza_array = df[['quantity', 'unit_price', 'total_price', 'pi

pizza_array[:5]

... array([[ 1, 13, 13,  1,  1],
           [ 1, 16, 16,  2,  2],
           [ 1, 18, 18,  3,  2],
           [ 1, 20, 20,  4,  2],
           [ 1, 16, 16,  5,  2]])
```

Keterangan:

Memotong kolom numerik penting yang ada di dataFrame menjadi Numpy.

BROADCASTING (Operasi Massal)

```
#Simulasi kenaikan harga 10%
unit_price = pizza_array[:, 1] * 1.10
unit_price[:5]

array([14.3, 17.6, 19.8, 22. , 17.6])

#Hitung ulang total_price setelah kenaikan harga
total_price = pizza_array[:, 0] * unit_price
total_price[:5]

... array([14.3, 17.6, 19.8, 22. , 17.6])
```

Keterangan:

Operasi massal di beberapa data yang penting.

UNIVERSAL FUNCTIONS (ufuncs)

```
#Statistik dasar
np.mean(qty), np.median(qty), np.std(qty), np.min(qty), np.max
...
(np.float64(1.0196215549156726),
 np.float64(1.0),
 np.float64(0.1430755379369698),
 np.int64(1),
 np.int64(4))

#Operasi matematika
np.sqrt(pizza_array[:, 2]) # akar dari total_price
np.log(pizza_array[:, 2] + 1)

array([2.63905733, 2.83321334, 2.94443898, ..., 2.56494936,
       3.04452244,
       2.56494936])
```

Keterangan:

Fungsi yang berbentuk secara universal.

2.4 Penanganan data hilang (missing values)

```
df.isna().sum()
...
0
pizza_id 0
order_id 0
pizza_name_id 0
quantity 0
order_date 0
order_time 0
unit_price 0
total_price 0
pizza_size 0
pizza_category 0
pizza_ingredients 0
```


Keterangan:

Cek missing value di dalam dataframe.

```
#menghapus tabel
df = df.dropna(subset=['price_check'])
```

Keterangan:

Melakukan penghapusan pada kolom price_check.

Tugas Praktikum:

1. Analisis dataset Pizza Sales untuk menentukan jenis dengan jumlah penjualan tertinggi dan tren harian penjualan.

```
#Jenis Pizza dengan Penjualan Tertinggi
top_quantity = (
    df.groupby('pizza_name')['quantity']
      .sum()
      .sort_values(ascending=False)
      .head(10)
)
print(top_quantity)
```

```
... pizza_name
the classic deluxe pizza      2453
the barbecue chicken pizza    2432
the hawaiian pizza            2422
the pepperoni pizza           2418
the thai chicken pizza        2371
the california chicken pizza   2370
the sicilian pizza             1938
the spicy italian pizza        1924
the southwest chicken pizza    1917
the big meat pizza             1914
Name: quantity, dtype: int64
```

```

#Berdasarkan Total Pendapatan
top_revenue = (
    df.groupby('pizza_name')['total_price']
      .sum()
      .sort_values(ascending=False)
      .head(10)
)
print(top_revenue)

```

```

... pizza_name
the thai chicken pizza      41656
the barbecue chicken pizza  40944
the california chicken pizza 39632
the classic deluxe pizza    37944
the spicy italian pizza     33592
the southwest chicken pizza 33268
the italian supreme pizza   32348
the hawaiian pizza          31183
the four cheese pizza       30576
the sicilian pizza          30456
Name: total_price, dtype: int64

```

```

#Tren Penjualan Harian
daily_sales = (
    df.groupby('order_date')['total_price']
      .sum()
      .reset_index()
)

```

```

daily_sales['day_name'] = daily_sales['order_date'].dt.day_name()

```

```

#Rata-rata Penjualan per Hari
day_trend = (
    daily_sales.groupby('day_name')['total_price']
      .mean()
      .reindex([
          'Monday', 'Tuesday', 'Wednesday',
          'Thursday', 'Friday', 'Saturday', 'Sunday'
      ])
)
print(day_trend)

```

```

... day_name
Monday      2173.937500
Tuesday     2134.307692
Wednesday   2138.711538
Thursday    2309.961538
Friday      2645.900000
Saturday    2303.461538
Sunday      1855.019231
Name: total_price, dtype: float64

```

2. Lakukan normalisasi terhadap kolom numerik menggunakan NumPy.

```
# Pilih kolom numerik
numerical_cols = ['quantity', 'unit_price', 'total_price']

# Salin data agar aman
df_norm = df.copy()

for col in numerical_cols:
    col_min = np.min(df[col])
    col_max = np.max(df[col])

    df_norm[col] = (df[col] - col_min) / (col_max - col_min)
```

df_norm[numerical_cols].head()

	quantity	unit_price	total_price
0	0.0	0.153846	0.056338
1	0.0	0.269231	0.098592
2	0.0	0.346154	0.126761
3	0.0	0.423077	0.154930
4	0.0	0.269231	0.098592

```
for col in numerical_cols:
    mean = np.mean(df[col])
    std = np.std(df[col])

    df_norm[col] = (df[col] - mean) / std
```

df_norm.head()

	pizza_id	order_id	pizza_name_id	quantity	order_date	ord
0	1	1	hawaiian_m	-0.137141	2015-01-01	
1	2	2	classic_dlx_m	-0.137141	2015-01-01	
2	3	2	five_cheese_l	-0.137141	2015-01-01	
3	4	2	ital_supr_l	-0.137141	2015-01-01	

Praktikum 3

Visualisasi Data dengan Matplotlib (CPMK M3)

Tujuan Praktikum:

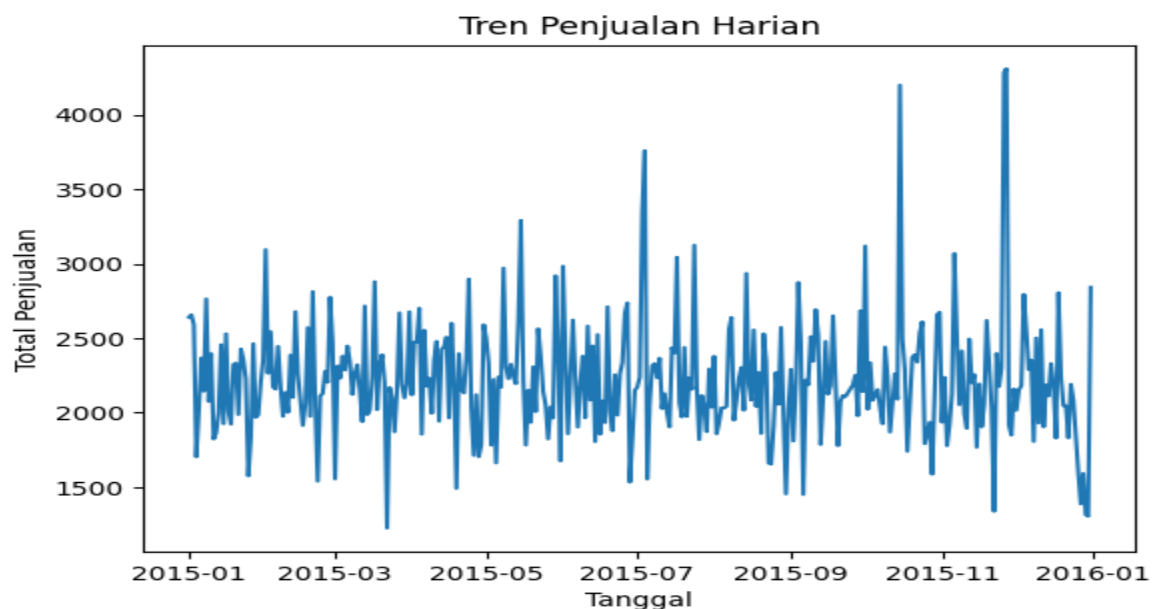
Mahasiswa mampu menyajikan data dalam bentuk visualisasi yang informatif untuk mendukung analisis dan pengambilan keputusan.

Materi Pokok:

1. Visualisasi dasar: *line plot*, *bar chart*, *scatter plot*, dan *histogram*
2. Kustomisasi grafik: judul, label sumbu, legenda
3. *Subplot* dan pengaturan tata letak
4. Visualisasi distribusi dan hubungan antarvariabel

3.1 Visualisasi dasar: *line plot*, *bar chart*, *scatter plot*, dan *histogram*

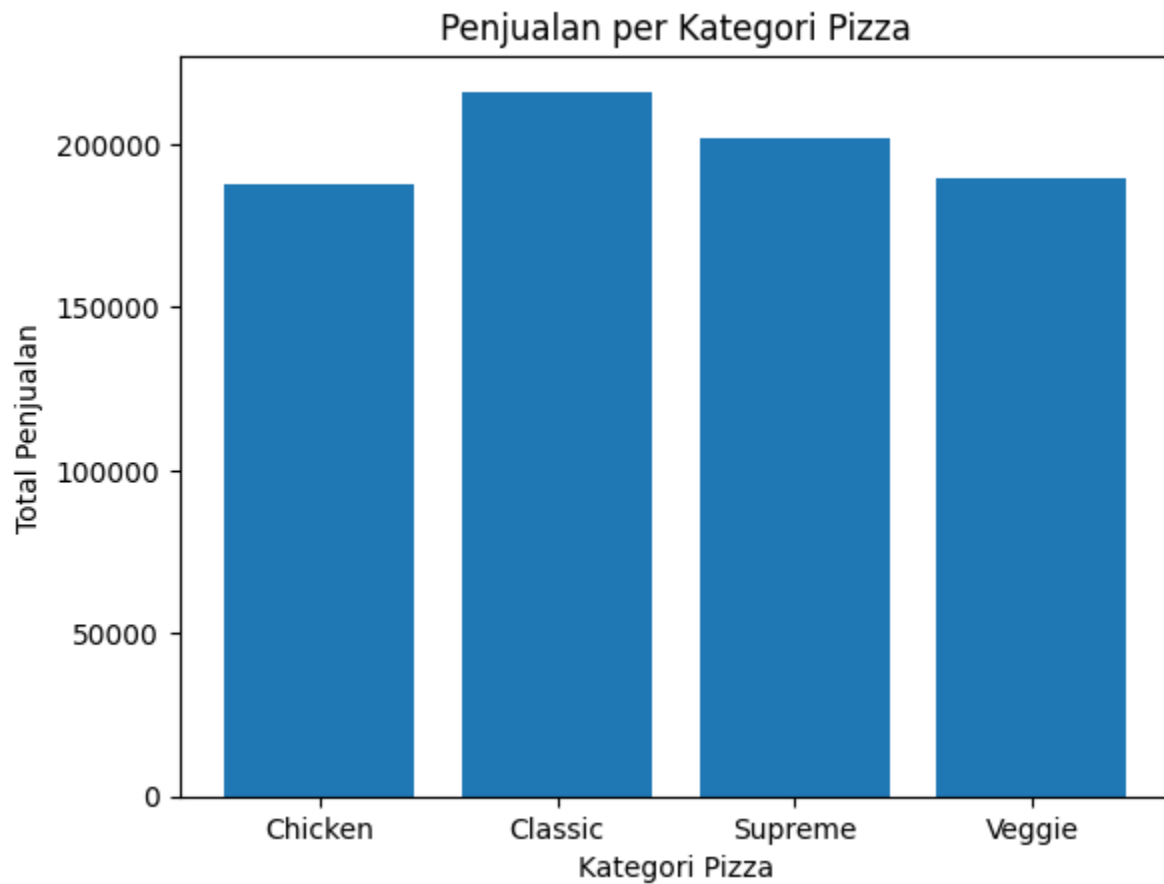
```
#Line Plot - Tren Penjualan Harian
daily_sales = df.groupby('order_date')['total_price'].sum()
plt.figure()
plt.plot(daily_sales.index, daily_sales.values)
plt.xlabel('Tanggal')
plt.ylabel('Total Penjualan')
plt.title('Tren Penjualan Harian')
plt.show()
```



Keterangan:

Visualisasi line plot untuk data order date dan total price.

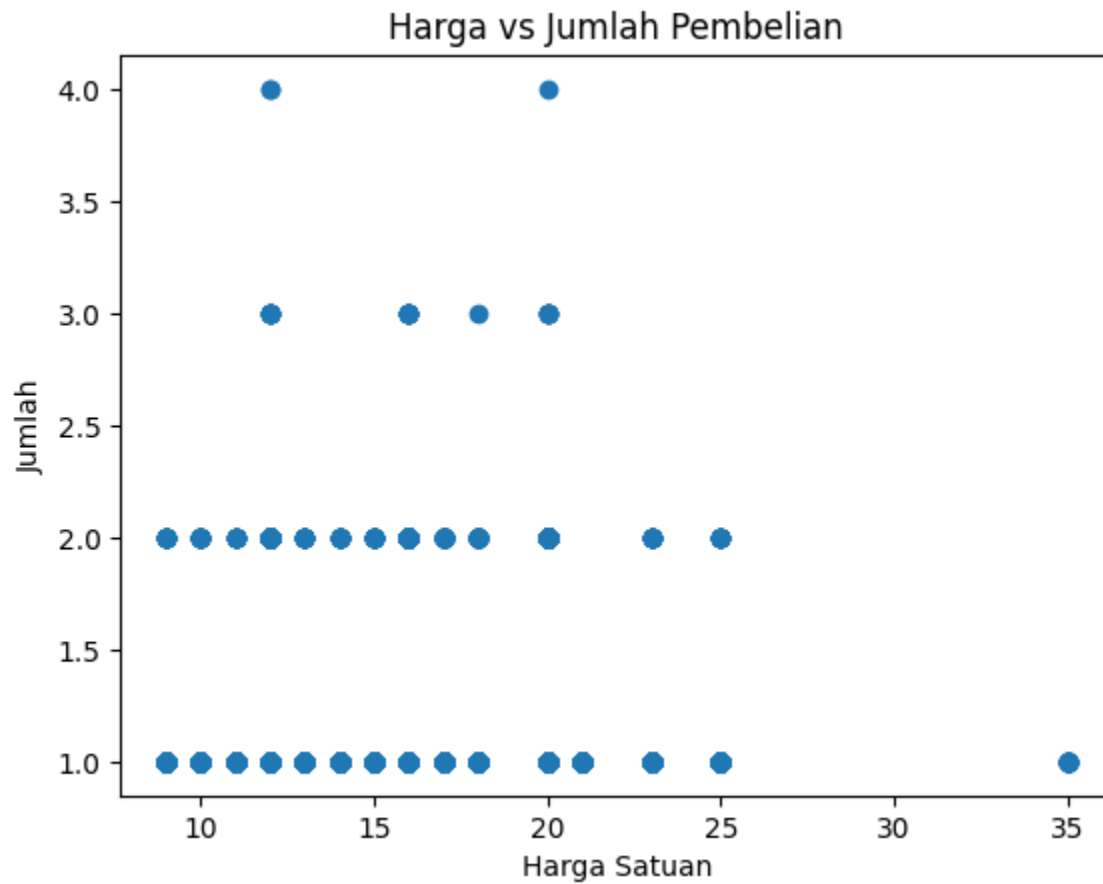
```
#Bar Chart - Penjualan per Kategori Pizza
category_sales = df.groupby('pizza_category')['total_price'].sum()
plt.figure()
plt.bar(category_sales.index, category_sales.values)
plt.xlabel('Kategori Pizza')
plt.ylabel('Total Penjualan')
plt.title('Penjualan per Kategori Pizza')
plt.show()
```



Keterangan:

Bar Chart penjualan perkategori pizza.

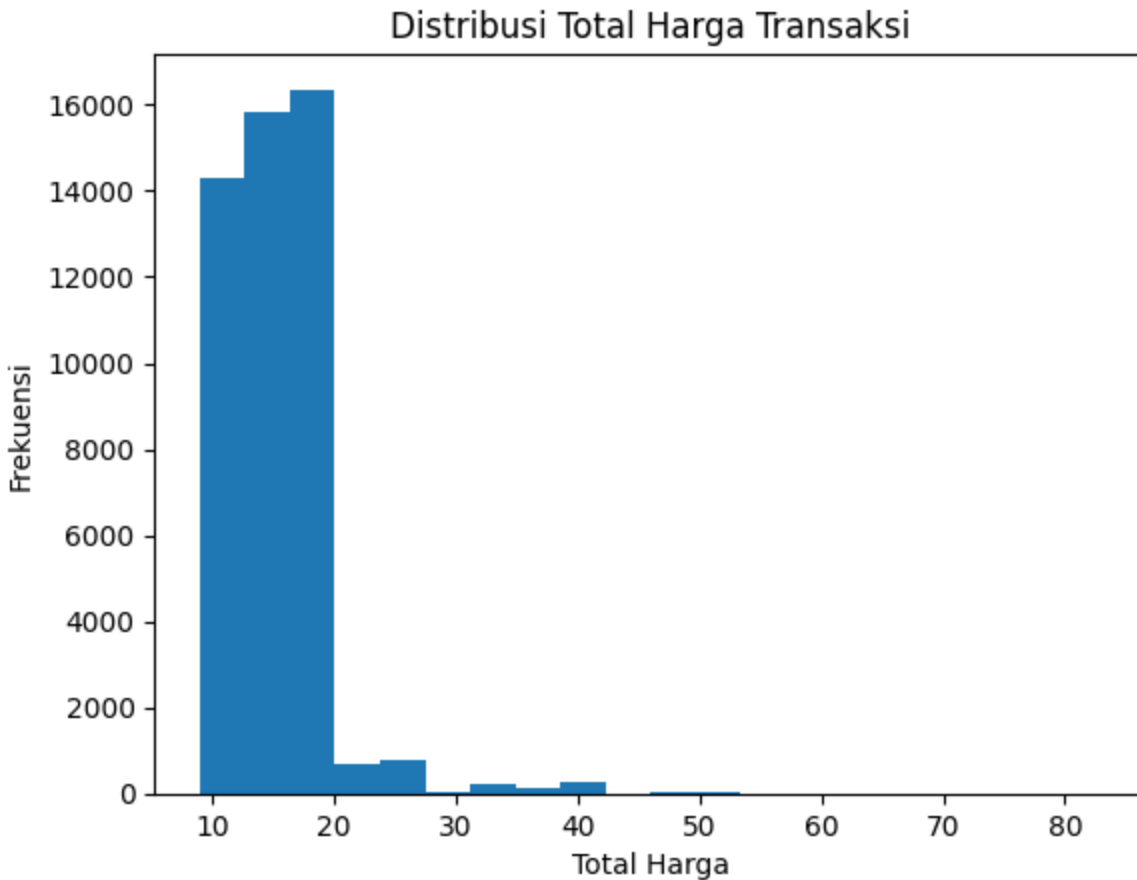
```
#Scatter Plot - Harga vs Jumlah
plt.figure()
plt.scatter(df['unit_price'], df['quantity'])
plt.xlabel('Harga Satuan')
plt.ylabel('Jumlah')
plt.title('Harga vs Jumlah Pembelian')
plt.show()
```



Keterangan:

Scater-Plot menggunakan data unit price dan quantity.

```
#Histogram - Distribusi Total Transaksi
plt.figure()
plt.hist(df['total_price'], bins=20)
plt.xlabel('Total Harga')
plt.ylabel('Frekuensi')
plt.title('Distribusi Total Harga Transaksi')
plt.show()
```

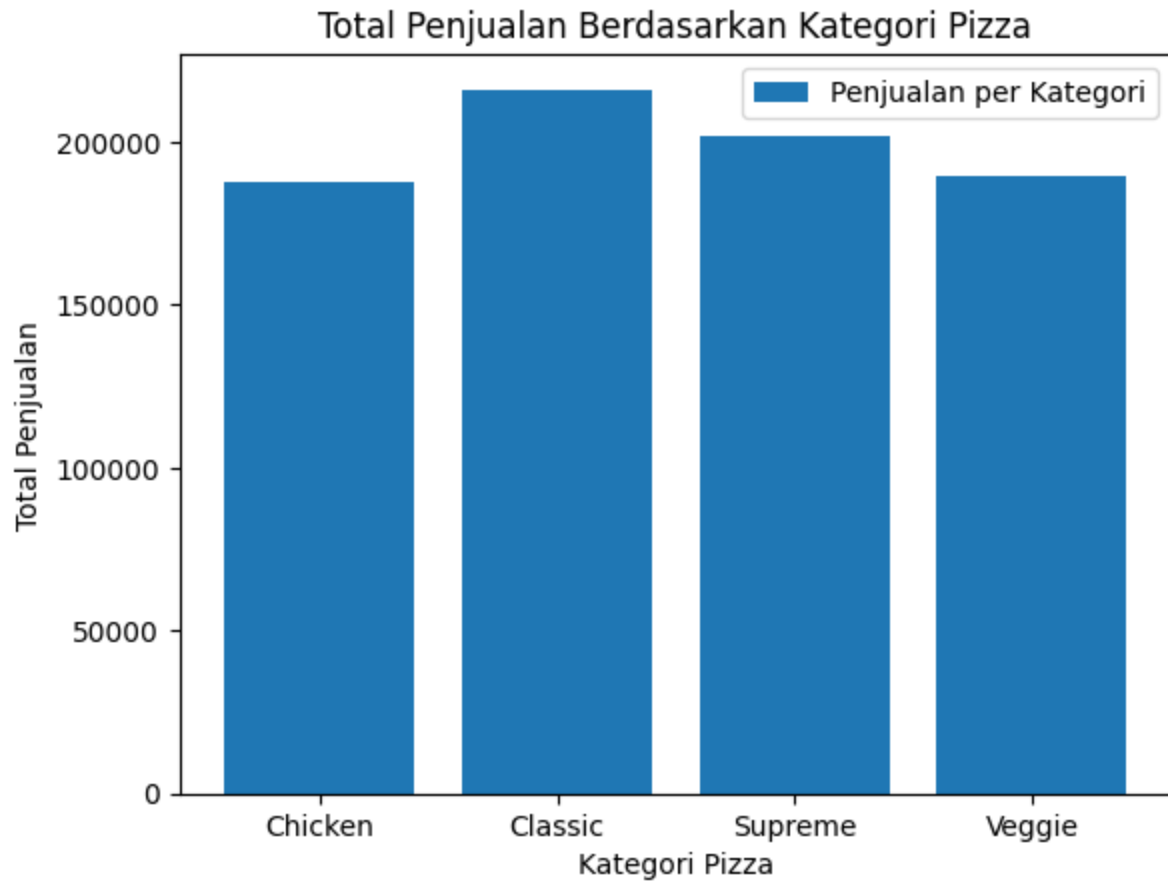


Keterangan:

Histogram menggunakan data total price.

3.2 Kustomisasi grafik: judul, label sumbu, legenda

```
plt.figure()  
plt.bar(category_sales.index, category_sales.values, label='Penjualan per Kategori')  
plt.title('Total Penjualan Berdasarkan Kategori Pizza')  
plt.xlabel('Kategori Pizza')  
plt.ylabel('Total Penjualan')  
plt.legend()  
  
plt.show()
```

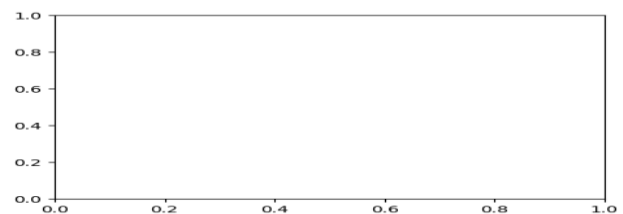
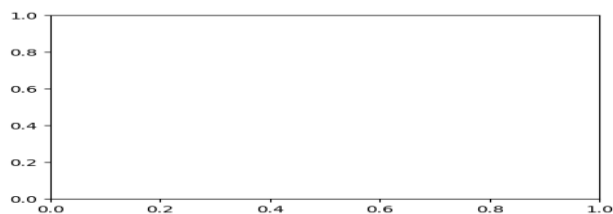


3.3 Subplot dan pengaturan tata letak

3.3 Subplot dan pengaturan tata letak



```
#Subplot memungkinkan beberapa grafik ditampilkan dalam satu figure
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
#fig, axes = plt.subplots(baris, kolom, figsize=(lebar, tinggi))
```



Keterangan:

Subplot memungkinkan beberapa grafik dalam satu figure.


```

▶ #Pengaturan Tata Letak (Layout)
plt.tight_layout()
plt.show()

... <Figure size 640x480 with 0 Axes>

▶ #plt.subplots_adjust() (Kontrol Manual)
plt.subplots_adjust(
    left=0.1,
    right=0.95,
    top=0.9,
    bottom=0.1,
    wspace=0.3,
    hspace=0.4
)

... <Figure size 640x480 with 0 Axes>

```

Keterangan:

Pengaturan tata letak dan kontrol manual pada visualisasi.

```

▶ #Judul Global (suptitle)
fig.suptitle('Analisis Penjualan Pizza', fontsize=14)
plt.tight_layout(rect=[0, 0, 1, 0.95])

... <Figure size 640x480 with 0 Axes>

```

Keterangan:

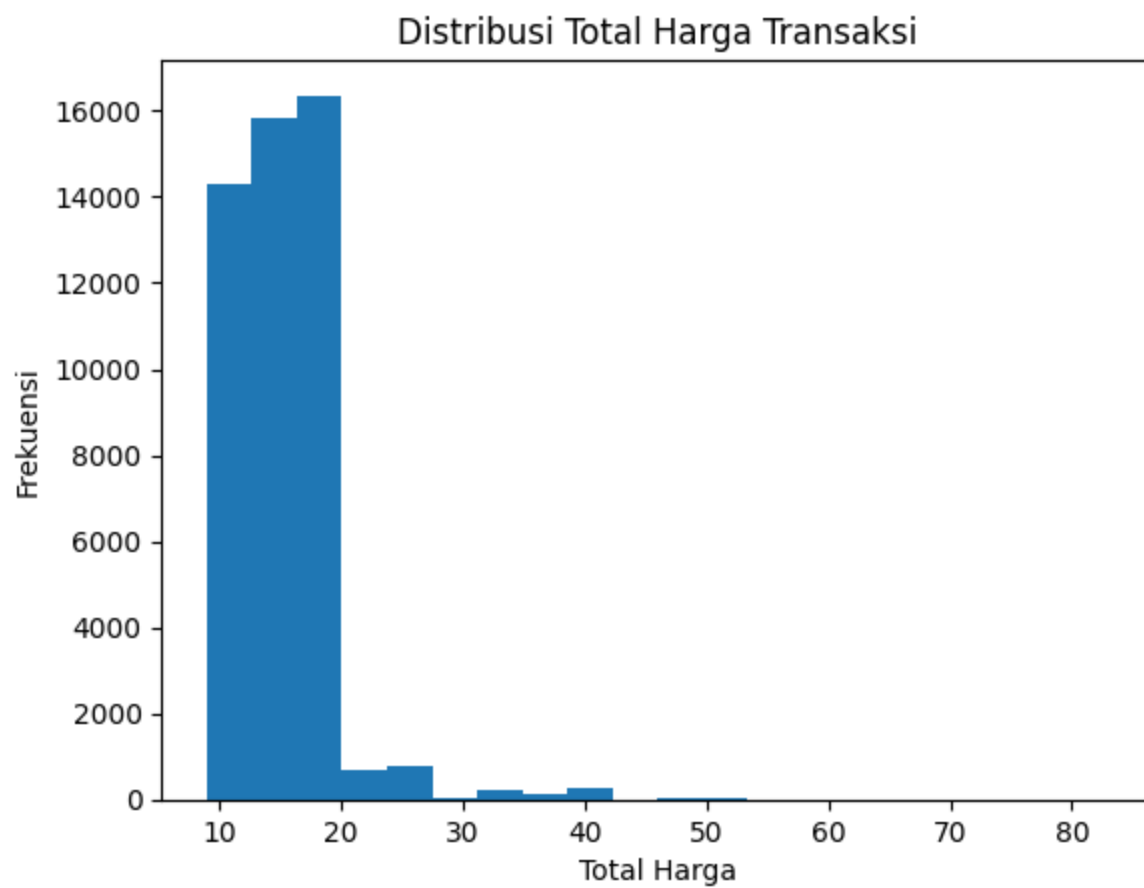
Judul global dari visualisasi data.

3.4 Visualisasi distribusi dan hubungan antarvariabel

```

#Histogram - Distribusi total_price
plt.figure()
plt.hist(df['total_price'], bins=20)
plt.title('Distribusi Total Harga Transaksi')
plt.xlabel('Total Harga')
plt.ylabel('Frekuensi')
plt.show()

```



Keterangan:

Visualisasi total price.

Tugas Praktikum:

1. Buat histogram distribusi jumlah pendapatan berdasarkan bulan.

1. Buat histogram distribusi jumlah pendapatan berdasarkan bulan.

```
df['order_date'] = pd.to_datetime(
    df['order_date'],
    format='mixed',
    dayfirst=True,
    errors='coerce'
)

# Hapus tanggal tidak valid
df = df.dropna(subset=['order_date'])

# Ekstrak bulan
df['month'] = df['order_date'].dt.month
```

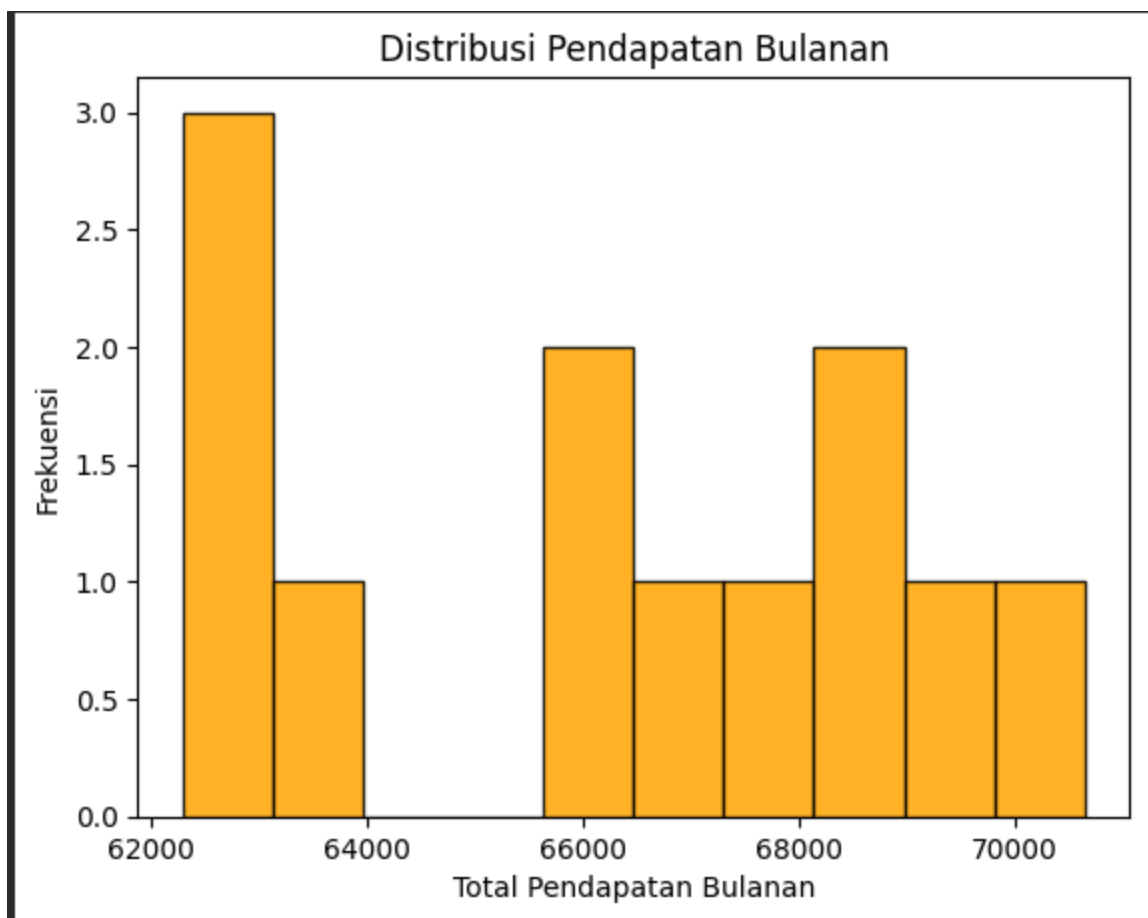
```
monthly_revenue = (
    df.groupby('month')['total_price']
    .sum()
)
```

```
monthly_revenue.index = monthly_revenue.index.map({
    1: 'Jan', 2: 'Feb', 3: 'Mar', 4: 'Apr', 5: 'Mei', 6: 'Jun',
    7: 'Jul', 8: 'Agu', 9: 'Sep', 10: 'Okt', 11: 'Nov', 12: 'Des'
})
```

```
plt.figure()
plt.hist(
    monthly_revenue.values,
    bins=10,
    color='orange',
    edgecolor='black',
    alpha=0.85
)

plt.title('Distribusi Pendapatan Bulanan')
plt.xlabel('Total Pendapatan Bulanan')
plt.ylabel('Frekuensi')

plt.show()
```



2. Buat *scatter plot* untuk menganalisis hubungan antara jumlah total penjualan dan kategori pizza.

2. Buat scatter plot untuk menganalisis hubungan antara jumlah total penjualan dan kategori pizza.

```
#agregasi data
# Ambil kategori unik
categories = df['pizza_category'].unique()

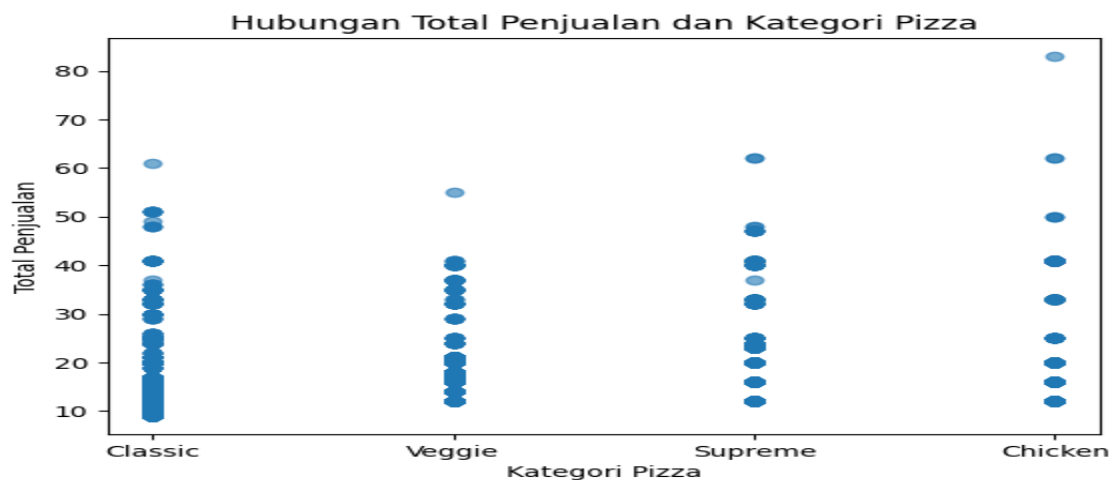
# Peta kategori ke angka
category_map = {cat: i for i, cat in enumerate(categories)}
df['category_num'] = df['pizza_category'].map(category_map)
```

```
plt.figure()
plt.scatter(
    df['category_num'],
    df['total_price'],
    alpha=0.6
)

plt.title('Hubungan Total Penjualan dan Kategori Pizza')
plt.xlabel('Kategori Pizza')
plt.ylabel('Total Penjualan')

# Label kategori di sumbu X
plt.xticks(
    ticks=range(len(categories)),
    labels=categories
)

plt.show()
```



Praktikum 4

Machine Learning dengan Scikit-Learn (CPMK M4)

Tujuan Praktikum:

Mahasiswa mampu membangun dan mengevaluasi model *machine learning* sederhana untuk kasus regresi dan klasifikasi.

Materi Pokok:

1. *Preprocessing* data: *scaling*, *encoding*, dan *train-test split*
2. Model dasar: *Linear Regression*, *Logistic Regression*, dan *K-Nearest Neighbor*
3. Evaluasi model: *accuracy*, *precision*, *recall*, dan *mean squared error*
4. *Cross-validation*

4.1 Preprosesing data: *scaling*, *encoding*, dan *train-test split*

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
```

Keterangan:

Import library yang diperlukan untuk preprosesing sampai pemodelan.

```
df = pd.read_excel('/content/drive/MyDrive/Semester7 2026/Data Science/pizza_sales_raw.xlsx', header=1)
#semua bentuk karakteristik data bisa di import seperti excel, csv, json
df.head() #melihat 5 baris data pertama
```

	pizza_id	order_id	pizza_name_id	quantity	order_date	order_time	unit_price	total_price	pizza_size
0	1.0	1.0	hawaiian_m	1.0	1/1/2015	11:38:36	13.25	13.25	M
1	2.0	2.0	classic_dlx_m	1.0	1/1/2015	11:57:40	16.00	16.00	M
2	3.0	2.0	five_cheese_l	1.0	1/1/2015	11:57:40	18.50	18.50	L
3	4.0	2.0	ital_supr_l	1.0	1/1/2015	11:57:40	20.75	20.75	L

Keterangan:

Load data yang mempunyai format excel.

```
#menghapus angka nul
df['pizza_id'] = df['pizza_id'].astype(int)
df['order_id'] = df['order_id'].astype(int)
df['quantity'] = df['quantity'].astype(int)

df['unit_price'] = df['unit_price'].astype(int)
df['total_price'] = df['total_price'].astype(int)

df['order_date'] = pd.to_datetime(
    df['order_date'],
    format='mixed',
    dayfirst=True
)
```

```
df.info()
df.head()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48620 entries, 0 to 48619
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   pizza_id              48620 non-null  int64
1   order_id              48620 non-null  int64
2   pizza_name_id         48620 non-null  object
3   quantity              48620 non-null  int64
4   order_date            48620 non-null  datetime64[ns]
5   order_time            48620 non-null  object
6   unit_price            48620 non-null  int64
7   total_price           48620 non-null  int64
8   pizza_size            48620 non-null  object
9   pizza_category        48620 non-null  object
10  pizza_ingredients     48620 non-null  object
11  pizza_name            48620 non-null  object
dtypes: datetime64[ns](1), int64(5), object(6)
memory usage: 4.5+ MB
```

Keterangan:

Data preprosesing dengan melakukan normalisasi data menjadi intenger.

```

# 4. FEATURE ENGINEERING
df['day_of_week'] = df['order_date'].dt.dayofweek
df['month'] = df['order_date'].dt.month
df['is_weekend'] = df['day_of_week'].isin([5, 6]).astype(int)

```

Keterangan:

Feature enggining untuk pemodelan dan categoru pemodelan.

4.2 Model dasar: RandomForestRegressor

```

X = df[
    [
        'quantity',
        'unit_price',
        'pizza_size',
        'pizza_category',
        'day_of_week',
        'month',
        'is_weekend'
    ]
]
y = df['total_price']

```

```

# 6. PREPROCESSING PIPELINE
categorical_features = ['pizza_size', 'pizza_category']
numeric_features = [
    'quantity',
    'unit_price',
    'day_of_week',
    'month',
    'is_weekend'
]

preprocessor = ColumnTransformer(
    transformers=[
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
        ('num', 'passthrough', numeric_features)
    ]
)

```

Keterangan:

Pemetaan data yang paling cocok untuk pemodelan model machine learning.

```
# 7. MODEL
model = RandomForestRegressor(
    n_estimators=200,
    random_state=42,
    n_jobs=-1
)

pipeline = Pipeline(
    steps=[
        ('preprocessor', preprocessor),
        ('model', model)
    ]
)
```

Keterangan:

Pemodelan menggunakan Random Forest Regression.

```
# 8. TRAIN-TEST SPLIT
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

Keterangan:

Splitting data dengan ketentuan 80:20.

```
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("=== MODEL EVALUATION ===")
print(f"MAE : {mae:.2f}")
print(f"RMSE : {rmse:.2f}")
print(f"R2 : {r2:.3f}")

...
=== MODEL EVALUATION ===
MAE : 0.00
RMSE : 0.08
R2 : 1.000
```

4.3 Evaluasi Model: *accuracy*, *precision*, *recall*, dan *mean squared error*

```
from sklearn.metrics import accuracy_score, precision_score, recall_score

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')

print("=== CLASSIFICATION EVALUATION ===")
print(f"Accuracy : {accuracy:.3f}")
print(f"Precision : {precision:.3f}")
print(f"Recall : {recall:.3f}")

... === CLASSIFICATION EVALUATION ===
Accuracy : 0.999
Precision : 0.998
Recall : 0.999
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565:
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

4.4 Cross-validation

```
cv_precision = cross_val_score(
    pipeline,
    X,
    y,
    cv=5,
    scoring='precision_weighted'
)

cv_recall = cross_val_score(
    pipeline,
    X,
    y,
    cv=5,
    scoring='recall_weighted'
)

print(f"Mean Precision : {cv_precision.mean():.3f}")
print(f"Mean Recall : {cv_recall.mean():.3f}")

... Mean Precision : 1.000
Mean Recall : 1.000
```

Tugas Praktikum:

1. Bandingkan performa beberapa model menggunakan *cross-validation*.

```
# 7. RANDOM FOREST MODEL
# =====
rf_model = RandomForestRegressor(
    n_estimators=200,
    random_state=42,
    n_jobs=-1
)

pipeline = Pipeline(
    steps=[
        ('preprocessor', preprocessor),
        ('model', rf_model)
    ]
)
```

```
# 8. TRAIN-TEST SPLIT
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

pipeline.fit(X_train, y_train)
```

```
y_pred = pipeline.predict(X_test)
```



11. EVALUATION

```
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

```
print("=== RANDOM FOREST REGRESSION EVALUATION ===")
print(f"MSE : {mse:.2f}")
print(f"RMSE : {rmse:.2f}")
print(f"MAE : {mae:.2f}")
print(f"R2 : {r2:.3f}")
```



```
=== RANDOM FOREST REGRESSION EVALUATION ===
MSE : 0.00
RMSE : 0.00
MAE : 0.00
R2 : 1.000
```



12. CROSS-VALIDATION

```
cv_scores = cross_val_score(
    pipeline,
    X,
    y,
    cv=5,
    scoring='neg_mean_squared_error'
)
```

```
cv_rmse = np.sqrt(-cv_scores)
```

```
print("\n=== CROSS-VALIDATION (5-FOLD) ===")
print(f"RMSE per fold : {cv_rmse}")
print(f"Mean RMSE : {cv_rmse.mean():.2f}")
print(f"Std RMSE : {cv_rmse.std():.2f}")
```



```
=== CROSS-VALIDATION (5-FOLD) ===
RMSE per fold : [0. 0. 0. 0. 0.]
Mean RMSE : 0.00
Std RMSE : 0.00
```

Praktikum 5

Proyek Integratif Sains Data (CPMK M5)

Tujuan Praktikum:

Mahasiswa mampu mengintegrasikan seluruh tahapan proses sains data dalam sebuah studi kasus nyata secara end-to-end.

Materi Pokok:

1. Metodologi CRISP-DM
2. Pipeline sains data: pengumpulan data → eksplorasi → *preprocessing* → pemodelan → evaluasi
3. Interpretasi hasil dan penyusunan laporan

Contoh Studi Kasus:

Analisis Sentimen Ulasan Aplikasi Traveloka di App Store

Alur Pengerjaan:

1. Scraping data
2. Load dataset
2. Exploratory Data Analysis (EDA)
3. Preprocessing data
4. Pembangunan dan perbandingan model
5. Evaluasi dan interpretasi hasil

Tugas Akhir:

1. Pilih dataset terbuka Scraping data di APP Store Aplikasi Traveloka
2. Buat notebook lengkap yang mencakup seluruh tahapan sains data.
3. Presentasikan hasil analisis dan insight yang diperoleh.

5.1 Scaraping Data

```
!pip install Sastrawi
from Sastrawi.Stemmer.StemmerFactory import StemmerFactory

Collecting Sastrawi
  Downloading Sastrawi-1.0.1-py2.py3-none-any.whl.metadata (909 bytes)
  Downloading Sastrawi-1.0.1-py2.py3-none-any.whl (209 kB)
    209.7/209.7 kB 5.4 MB/s eta 0:00:00
Installing collected packages: Sastrawi
Successfully installed Sastrawi-1.0.1

import pandas as pd
import re
from wordcloud import WordCloud
import nltk
import requests
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
```

Keterangan:

Import library yang dibutuhkan dala expolasi data, preprosesing dan pemodelan.

SerpAPI (Scrapping)

```
!pip install serpapi

... Collecting serpapi
  Downloading serpapi-0.1.5-py2.py3-none-any.whl.metadata (10 kB)
  Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packag
  Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/pythor
  Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-pa
  Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/c
  Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/c
  Downloading serpapi-0.1.5-py2.py3-none-any.whl (10 kB)
  Installing collected packages: serpapi
  Successfully installed serpapi-0.1.5

import serpapi
import os

from dotenv import load_dotenv

api_key = "131ab5b32da04cf406eed84d858f5fd6af16f0ff93a2e3868e2b4e868d413f95"
```

Keterangan:

Scaping data menggunakan SerAPI dengan menggunakan API key yang di dapatkan untuk meminta data yang berada di app store.

```

▶ result = client.search(
    engine="google_play_product",
    product_id="com.traveloka.android",  # ID aplikasi
    store="apps",
    all_reviews = True,
    num=199,                          # jumlah review per page (maks 199)
    hl="id",                          # bahasa (id = Indonesia, en = English)
)

▶ dataReviews = []

for i in result.get("reviews"):
    if i['iso_date'][:4] in ["2023", "2024", "2025"]:
        dataReviews.append(i)

```

Keterangan:

Kriteria data yang akan di minta dari app store dan parameter tambahan yang digunakan.

```

▶ result.get('reviews')[0]
... {'id': '7d72c2d6-3aa5-461c-8672-8669a403319a',
    'title': 'Pengguna Google',
    'avatar': 'https://play-lh.googleusercontent.com/EGemoI2NTXmTsBVtJqk8jxF9rh8ApRWfsIMQSt2uE40cpQqbFu7f7NbTK051x80nuSijCz7sc3a277R67g',
    'rating': 5.0,
    'snippet': 'terima kasih yang sebesar besarnya. karena aplikasi ini sangat membantu keuangan dalam membeli dan mencari tiket. sangat bagus. memberikan solusi dengan egektif dan tepat. semua pelayanan ramah dan sangat baik. saya berharap traveloka selalu terdepan dan tetap no 1 dihati masyarakat. dan masih banyak lagi kejutan2 untuk kedepannya. semoga selalu sukses dan berjaya sampai kapanpun. terima kasih sy yang sangat besar buat traveloka 🙏🙏',
    'likes': 0,
    'date': 'February 05, 2019',
    'iso_date': '2019-02-05T01:50:08Z'}

dataReviews[0]['iso_date'][:4] in ["2025"]

True

df = pd.DataFrame(dataReviews)

```

Keterangan:

Hasil data yang didapatkan dan mentimpan data tersebut ke dataframe, untuk dilakukan langkah selanjutnya.

```

df.to_csv("dataset_traveloka(2023-2025).csv", index = False)

```

Keterangan:

Menyimpan data tersebut ke dalam format csv biar lebih mudah untuk melakukan pengolahan data.

5.2 Load Dataset

Pengumpulan Data

```
df = pd.read_csv("/content/drive/MyDrive/Semester7 2026/Pemrosesan Teks/Analisis Sentimen App Traveloka di Play Store/traveloka.csv")
df
```

	rating	snippet	likes	sentiment
0	1	MENDADAK LIMIT TURUN SAMPAI DI 0 padahal aktif...	24	Negatif
1	1	Sangat kecewa dengan Traveloka. Tiket saya dib...	11	Negatif
2	1	Proses verifikasi payment gajelas, udah paymen...	16	Negatif
3	1	untuk menu penjualan tiket dan akomodasinya us...	18	Netral
4	1	poin yg dapat dr tiap beli tiket terlalu banya...	10	Positif
...	...	...	...	...
1388	5	sangat baik cara penyampaianannya	0	Positif
1389	5	Proses transaksi mudah dan lancar	0	Positif
1390	5	sangat membantu sekali sejak adanya paylater	0	Positif

Keterangan:

Data sudah dilakukan import dan akan dilakukan langkah langkah selanjutnya.

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1393 entries, 0 to 1392
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   rating      1393 non-null   int64
 1   snippet     1393 non-null   object
 2   likes       1393 non-null   int64
 3   sentiment   1393 non-null   object
dtypes: int64(2), object(2)
memory usage: 43.7+ KB

df = df.drop(['rating', 'likes'], axis = 1)

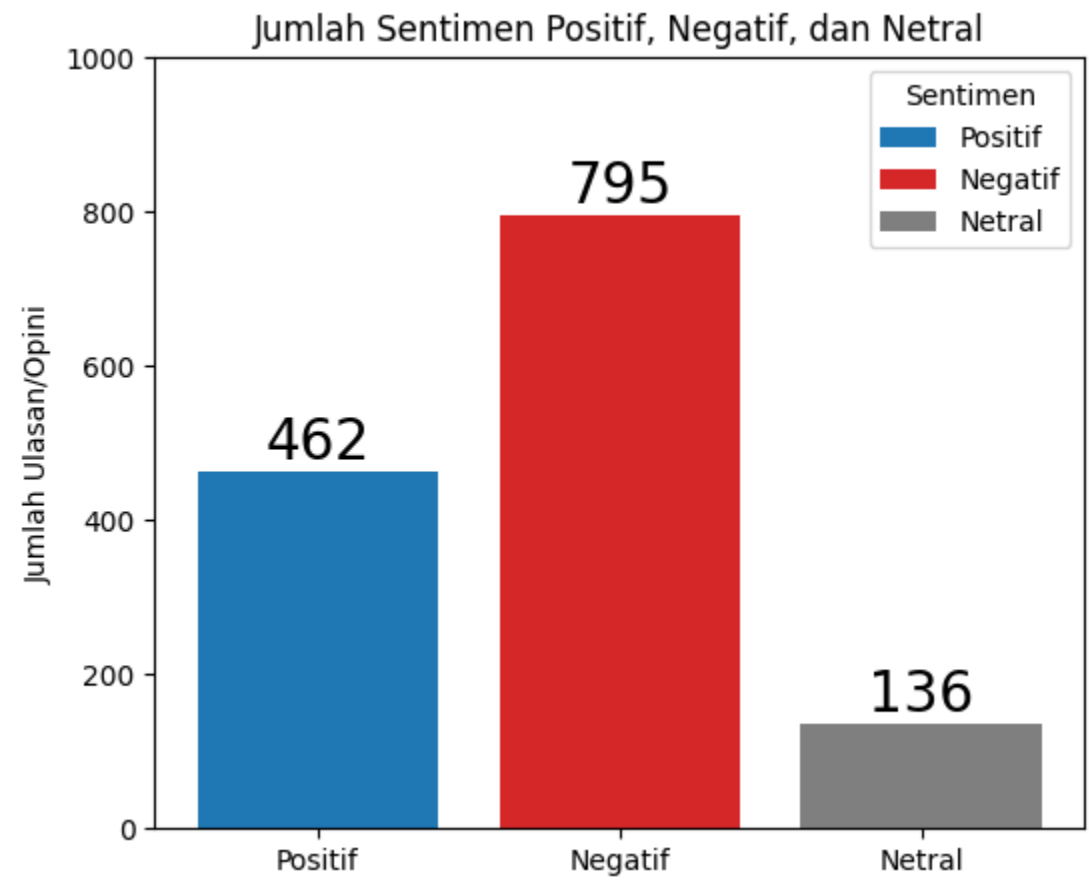
total_negatif = len(df[df['sentiment'] == "Negatif"])
total_positif = len(df[df['sentiment'] == "Positif"])
total_netral = len(df[df['sentiment'] == "Netral"])
```


Keterangan:

Explorasi data yang dilakukan dan melihat karakteristik data seperti apa.

5.3 Exploratory Data Analysis (EDA)

```
fig, ax = plt.subplots(figsize=(6,5))
fruits = ['Positif', 'Negatif', 'Netral']
counts = [total_positif, total_negatif, total_netral]
bar_labels = fruits
bar_colors = ['tab:blue', 'tab:red', 'tab:gray']
bar_container = ax.bar(fruits, counts, label=bar_labels, color=bar_colors)
ax.bar_label(bar_container, size = 20)
ax.set_ylim(0, 1000)
ax.set_ylabel('Jumlah Ulasan/Opini')
ax.set_title('Jumlah Sentimen Positif, Negatif, dan Netral')
ax.legend(title='Sentimen')
plt.show()
```



Keterangan:

Dataset yang digunakan pada penelitian ini berjumlah 1.392 data yang terdiri dari tiga kelas sentimen, yaitu negatif, netral, dan positif. Setiap data berisi teks ulasan/opini yang telah melalui tahap pra-pemrosesan sebelum digunakan pada proses klasifikasi.

Hasil analisis menunjukkan bahwa dataset bersifat tidak seimbang (imbalanced), dengan jumlah data sentimen negatif dan positif jauh lebih banyak dibandingkan sentimen netral. Kondisi ini berpotensi memengaruhi kinerja model klasifikasi, khususnya pada kelas netral.

Ringkasan distribusi data:

- Sentimen Negatif : dominan
- Sentimen Positif : cukup banyak
- Sentimen Netral : paling sedikit

5.4 Preprocessing Data

Pembersihan

```
url_pattern = re.compile(r'https?://\S+|www\.\S+')
def bersihkan_teks(teks):
    teks = re.sub(r'@\w+', ' ', teks) # Menghapus mention/username
    teks = url_pattern.sub(' ', teks) # Menghapus url/link
    teks = re.sub(r'[Tt][Tt]', ' kecewa ', teks) # Jika terdapat T_T dalam teks, dikonversi jadi kata kecewa
    teks = re.sub(r'[>][::]', ' kesal ', teks) # Jika terdapat >:( dalam teks, dikonversi jadi kata kesal
    teks = re.sub(r'[Bb][Ii][Nn][Tt][Aa][Nn][Gg]\s1', ' bintang satu ', teks) # Jika terdapat bintang 1, dikonversi jadi kata bintang satu
    teks = re.sub(r'[Bb][Ii][Nn][Tt][Aa][Nn][Gg]\s2', ' bintang dua ', teks) # Jika terdapat bintang 1, dikonversi jadi kata bintang dua
    teks = re.sub(r'[Bb][Ii][Nn][Tt][Aa][Nn][Gg]\s3', ' bintang tiga ', teks) # Jika terdapat bintang 1, dikonversi jadi kata bintang tiga
    teks = re.sub(r'[Bb][Ii][Nn][Tt][Aa][Nn][Gg]\s4', ' bintang empat ', teks) # Jika terdapat bintang 1, dikonversi jadi kata bintang empat
    teks = re.sub(r'[Bb][Ii][Nn][Tt][Aa][Nn][Gg]\s5', ' bintang lima ', teks) # Jika terdapat bintang 1, dikonversi jadi kata bintang lima
    teks = re.sub(r'^a-zA-Z\s', ' ', teks) # Hapus angka, simbol, hastag, mention
    teks = re.sub(r'\s+', ' ', teks) # Ganti spasi berlebih dengan satu spasi
    teks = teks.strip() # Hapus spasi di awal/akhir
    teks = teks.lower() # Semua huruf dijadikan huruf kecil
    return teks

df['snippet'] = df['snippet'].apply(lambda x: bersihkan_teks(x))
```

Keterangan:

Tahap ini dilakukan pembersihan data dari berbagai macam noise. Kode pada gambar digunakan untuk melakukan pra-pemrosesan (text preprocessing) sebelum data teks digunakan dalam proses analisis sentimen dan klasifikasi. Tujuan utama tahap ini adalah membersihkan teks dari noise dan menyeragamkan format teks agar mudah diproses oleh algoritma machine learning.

Tokenizing

```
df['tokenize'] = df['snippet'].apply(lambda x: x.split())
```

```
df
```

	snippet	sentiment	tokenize
0	mendadak limit turun sampai di padahal aktif d...	Negatif	[mendadak, limit, turun, sampai, di, padahal, ...
1	sangat kecewa dengan traveloka tiket saya diba...	Negatif	[sangat, kecewa, dengan, traveloka, tiket, say...
2	proses verifikasi payment gajelas udah payment...	Negatif	[proses, verifikasi, payment, gajelas, udah, p...

Keterangan:

Tokenisasi dilakukan untuk memisah teks per teks pada di setiap dokumen.

Normalisasi kata

```
dict_normal = dict(
    zip(df_normal['awal'], df_normal['normalisasi'])
)
```

```
def normalize_text(tokens):
    return [dict_normal.get(token, token) for token in tokens]
```

```
df['normalisasi'] = df['tokenize'].apply(normalize_text)
```

Keterangan:

Normalisasi dalam konteks *text preprocessing* adalah proses menyeragamkan kata/teks agar berbagai variasi penulisan yang maknanya sama diperlakukan sebagai satu bentuk yang konsisten oleh model.

Lematisasi

```

def stem_word(word):
    """Menghapus awalan dan akhiran umum Bahasa Indonesia secara sederhana"""
    prefixes = ['meng', 'meny', 'men', 'mem', 'me',
                'peng', 'peny', 'pen', 'pem', 'di', 'ke', 'se',
                'ber', 'ter', 'per']
    suffixes = ['kan', 'an', 'i', 'lah', 'nya', 'ku', 'mu']
    for p in prefixes:
        if word.startswith(p) and len(word) > len(p) + 2:
            word = word[len(p):]
            break
    for s in suffixes:
        if word.endswith(s) and len(word) > len(s) + 2:
            word = word[:-len(s)]
            break
    return word

def stem_sentence(sentence):
    return " ".join([stem_word(w) for w in sentence.split()])

df['lemmatisasi'] = df['normalisasi'].astype(str).apply(stem_sentence)

```

Keterangan:

Lematisasi adalah proses mengubah kata ke bentuk kata dasar (lemma) yang benar secara linguistik, sehingga makna kata tetap terjaga. Dalam analisis sentimen, lemmatisasi membantu model memahami bahwa berbagai bentuk kata sebenarnya memiliki makna yang sama.

Stopwords

```

from nltk.corpus import stopwords
nltk.download('stopwords')

... [nltk_data] Downloading package stopwords to /root/nltk_data...
... [nltk_data] Unzipping corpora/stopwords.zip.
True

stop_words = set(stopwords.words('indonesian'))
stop_list = ["waaa", "dong", "sih", "wkwwkw", "ah", "deh", "kok", "ny", "nya",
             "pun", "wkwk", "lah", "sll", "donk", "yaa", "doank", "hehehe", "oya",
             "dech", "nich", "ehh", "ya", "ribu", "a"]
remove_stop_list = ["kurang", "masalah", "lama", "jauh", "tapi", "keterlaluan",
                    "baik"]
stop_words.update(stop_list)
for i in remove_stop_list:
    stop_words.remove(i)

df['stopwords'] = df['normalisasi'].apply(lambda tokens: [word for word in tokens if word not in stop_words])

```

Keterangan:

Stopword removal adalah proses menghapus kata-kata umum yang sering muncul tetapi tidak membawa makna sentimen atau informasi penting dalam teks. Ini adalah salah satu tahap penting dalam text preprocessing setelah tokenisasi, normalisasi, dan lemmatisasi.

```

Stemming

factory = StemmerFactory()
stemmer = factory.create_stemmer()

custom_stem = {}

for awal_kata, stem_kata in zip(df_cus_stem_list['awal'], df_cus_stem_list['stemming']):
    if awal_kata not in custom_stem:
        custom_stem[awal_kata] = stem_kata

def safe_stem(word):
    if word in custom_stem:
        return custom_stem[word]
    return stemmer.stem(word)

safe_stem("mengurangi")
'kurang'

df['stemming'] = df['stopwords'].apply(lambda x: [safe_stem(word) for word in x])

```

Keterangan:

Stemming merupakan proses mengubah kata berimbuhan menjadi bentuk dasar dengan cara menghilangkan imbuhan. Proses ini bertujuan untuk mengurangi variasi kata dan meningkatkan konsistensi fitur teks sehingga dapat meningkatkan kinerja model analisis sentimen.

5.5 Pembangunan dan Perbandingan Model

```

Pembagian dataset

x = df['stemming'].to_numpy()
y = df['sentiment_encoded'].to_numpy()

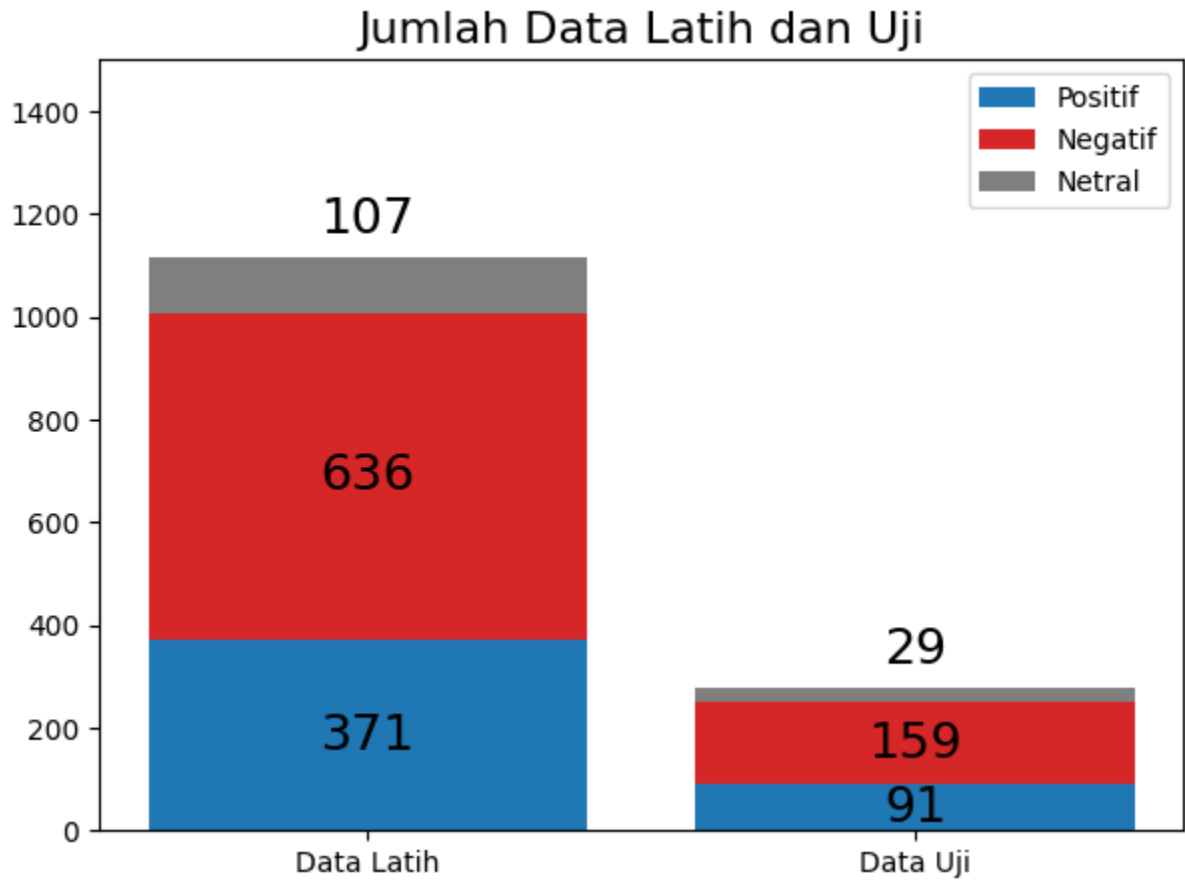
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)

sentiment_counts = {
    'Positif': np.array([0, 0]),
    'Negatif': np.array([0, 0]),
    'Netral': np.array([0, 0])
}

```

Keterangan:

Pembagian dataset yang akan dilakukan sebelum melakukan pemodelan yaitu 80:20 yang masing masingnya 80 data training dan 20 data testing.



Keterangan:

Visualisasi data yang didapatkab setelah melakukan spliting data secara keseluruhan.

Ekstraksi Fitur (TF-IDF)

```
tfidf = TfidfVectorizer(
    tokenizer=lambda x: x,
    preprocessor=lambda x: x,
    token_pattern=None
)

tfidf.fit(X_train)
```

...

TfidfVectorizer

TfidfVectorizer(preprocessor=<function <lambda> at 0x7aae3dada7a0>,
token_pattern=None,
tokenizer=<function <lambda> at 0x7aae3dad380>)

```
tfidf.get_feature_names_out()

array(['abal', 'absolute', 'ac', ..., 'zaman', 'zona', 'zoo'],
      dtype=object)
```

Keterangan:

TF-IDF (Term Frequency – Inverse Document Frequency) adalah metode feature extraction untuk mengubah teks menjadi nilai numerik berdasarkan tingkat kepentingan kata dalam sebuah dokumen dan seluruh kumpulan dokumen.

5.6 Evaluasi dan Interpretasi Hasil

1. Klasifikasi Menggunakan Decision Tree

Decision Tree adalah algoritma klasifikasi yang bekerja dengan membentuk struktur pohon keputusan, di mana setiap node merepresentasikan fitur, cabang merepresentasikan aturan keputusan, dan leaf node merepresentasikan kelas hasil prediksi.

Pada analisis sentimen, Decision Tree digunakan untuk mengklasifikasikan teks ke dalam kelas negatif, netral, dan positif berdasarkan fitur hasil ekstraksi teks.

```

from sklearn.tree import DecisionTreeClassifier, export_graphviz
from sklearn.model_selection import GridSearchCV

param_grid = {
    'max_depth': [10, 12, 14, 18, 19, 20],
    'min_samples_split': [4, 5, 6, 8, 10],
    'criterion': ['gini', 'entropy']
}

dt = DecisionTreeClassifier(random_state=42)

grid = GridSearchCV(
    estimator=dt,
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)
grid.fit(X_train, y_train)

print("Best Parameters:", grid.best_params_)
print("Best CV Score:", grid.best_score_)
print("Best Estimator:", grid.best_estimator_)

Best Parameters: {'criterion': 'gini', 'max_depth': 19, 'min_samples_split': 4}
Best CV Score: 0.7567244374419262
Best Estimator: DecisionTreeClassifier(max_depth=19, min_samples_split=4, random_state=42)

```

Keterangan:

Melakukan pencarian untuk parameter dan tuning terbaik.

```

model_decision_tree = DecisionTreeClassifier(
    criterion="entropy", random_state=0, max_depth=1, min_samples_split = 12
)

model_decision_tree.fit(X_train, y_train)

```

▼ **DecisionTreeClassifier** ⓘ ?

DecisionTreeClassifier(criterion='entropy', max_depth=1, min_samples_split=12, random_state=0)

Keterangan:

Model terbaik untuk data seperti ini terlihat seperti gambar di atas.


```

# Evaluasi
print("Akurasi:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, target_names=label_classes))

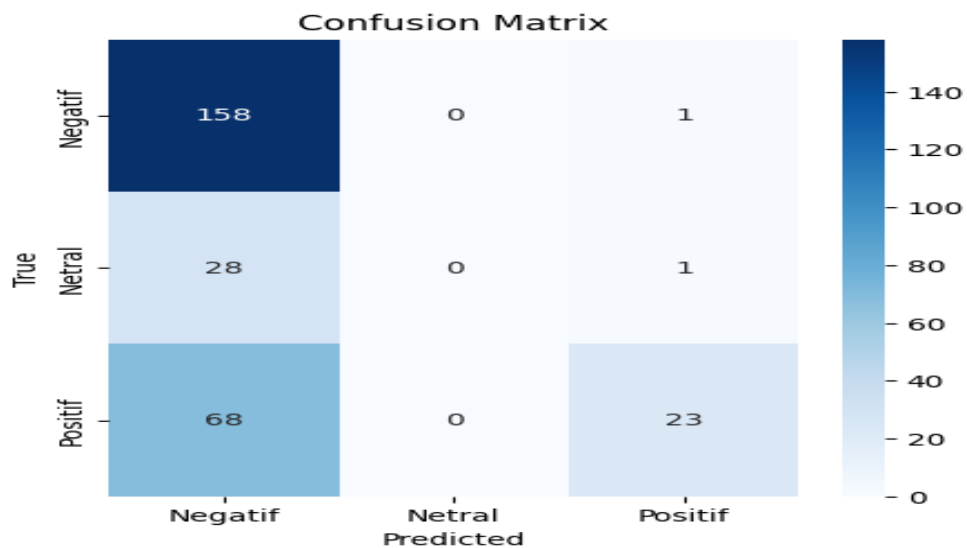
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(5,5))
sns.heatmap(cm, annot=True, fmt='d',
            xticklabels=label_classes,
            yticklabels=label_classes,
            cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix")
plt.show()

```

```
... Akurasi: 0.6487455197132617
```

```
Classification Report:
```

	precision	recall	f1-score	support
Negatif	0.62	0.99	0.77	159
Netral	0.00	0.00	0.00	29
Positif	0.92	0.25	0.40	91
accuracy			0.65	279
macro avg	0.51	0.42	0.39	279
weighted avg	0.65	0.65	0.57	279



Keterangan:

Confusion matrix tersebut menunjukkan performa model dalam mengklasifikasikan sentimen ke dalam tiga kategori, yaitu positif, netral, dan negatif.

- a. Untuk kelas negatif, model mengklasifikasikan 197 data negatif secara benar. Namun, masih terdapat 40 data yang diklasifikasikan salah, yaitu 29 diklasifikasikan positif dan 11 diklasifikasikan netral.
- b. Untuk kelas Netral, model mengklasifikasikan hanya 1 data netral secara benar. Terdapat total 40 data diklasifikasikan salah, yaitu 26 diklasifikasikan negatif dan 14 data diklasifikasikan positif.
- c. Pada kelas positif, model berhasil mengklasifikasikan 116 data dengan benar. Namun, masih terdapat 24 data yang diklasifikasikan salah, yaitu 22 diklasifikasikan negatif dan 2 diklasifikasikan netral.

Pada akurasi pengujian, model mencapai akurasi 75.12%, menunjukkan bahwa model mampu bekerja dengan baik pada data uji. Kinerja terbaik diperoleh pada kelas positif dan negatif dengan F1 score masing-masing sebesar 78% dan 82%, sedangkan kelas netral memiliki performa terendah dengan F1 score 4%, karena jumlah data netral yang sangat sedikit sehingga model lebih dominan pada prediksi kelas positif dan negatif.

2. Klasifikasi Menggunakan Suport Vector Machine (SVM)

Support Vector Machine (SVM) adalah algoritma klasifikasi yang bekerja dengan mencari hyperplane terbaik yang dapat memisahkan data ke dalam kelas-kelas yang berbeda dengan margin maksimum. Pada analisis sentimen, SVM digunakan untuk mengklasifikasikan teks ke dalam tiga kelas sentimen, yaitu negatif, netral, dan positif, berdasarkan fitur numerik hasil ekstraksi teks.

SVM sangat cocok digunakan untuk data teks karena mampu menangani data berdimensi tinggi dan sparse, terutama ketika dikombinasikan dengan metode ekstraksi fitur seperti TF-IDF.

```

from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from imblearn.over_sampling import SMOTE

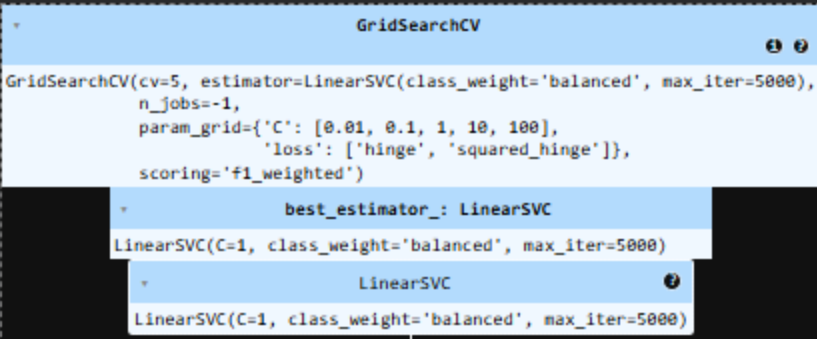
param_grid = {
    'C': [0.01, 0.1, 1, 10, 100],
    'loss': ['hinge', 'squared_hinge']
}

svm = LinearSVC(
    class_weight='balanced',
    max_iter=5000
)

grid = GridSearchCV(
    svm,
    param_grid,
    cv=5,
    scoring='f1_weighted',
    n_jobs=-1
)

grid.fit(X_train_smote, y_train_smote)

```



```

GridSearchCV
GridSearchCV(cv=5, estimator=LinearSVC(class_weight='balanced', max_iter=5000),
  n_jobs=-1,
  param_grid={'C': [0.01, 0.1, 1, 10, 100],
    'loss': ['hinge', 'squared_hinge']},
  scoring='f1_weighted')
  best_estimator_: LinearSVC
    LinearSVC(C=1, class_weight='balanced', max_iter=5000)
    LinearSVC
      LinearSVC(C=1, class_weight='balanced', max_iter=5000)

```

Keterangan:

Proses fine-tuning dan pelatihan model Support Vector Machine (SVM) menggunakan LinearSVC, SMOTE, dan GridSearchCV untuk tugas klasifikasi sentimen teks. Proses optimasi model Support Vector Machine (SVM) menggunakan GridSearchCV dengan data hasil oversampling SMOTE. Hasil tuning menghasilkan model terbaik LinearSVC dengan parameter $C=1$ dan `class_weight='balanced'`, yang mampu meningkatkan kinerja klasifikasi terutama pada kelas minoritas (netral).

```

▶ print("Best Parameters:", grid.best_params_)
print("Best CV Score:", grid.best_score_)
print("Best Estimator:", grid.best_estimator_)

... Best Parameters: {'C': 1, 'loss': 'squared_hinge'}
Best CV Score: 0.9405056744499524
Best Estimator: LinearSVC(C=1, class_weight='balanced', max_iter=5000)

model_svm = LinearSVC(
    C=1.0,                    # setara dengan kontrol kompleksitas model
    loss='squared_hinge',    # fungsi loss
    class_weight='balanced',  # mengatasi data tidak seimbang
    max_iter=5000,           # batas iterasi konvergensi
    random_state=42
)

model_svm.fit(X_train_smote, y_train_smote)

```

▼ LinearSVC
ⓘ ?

LinearSVC(class_weight='balanced', max_iter=5000, random_state=42)

Keterangan:

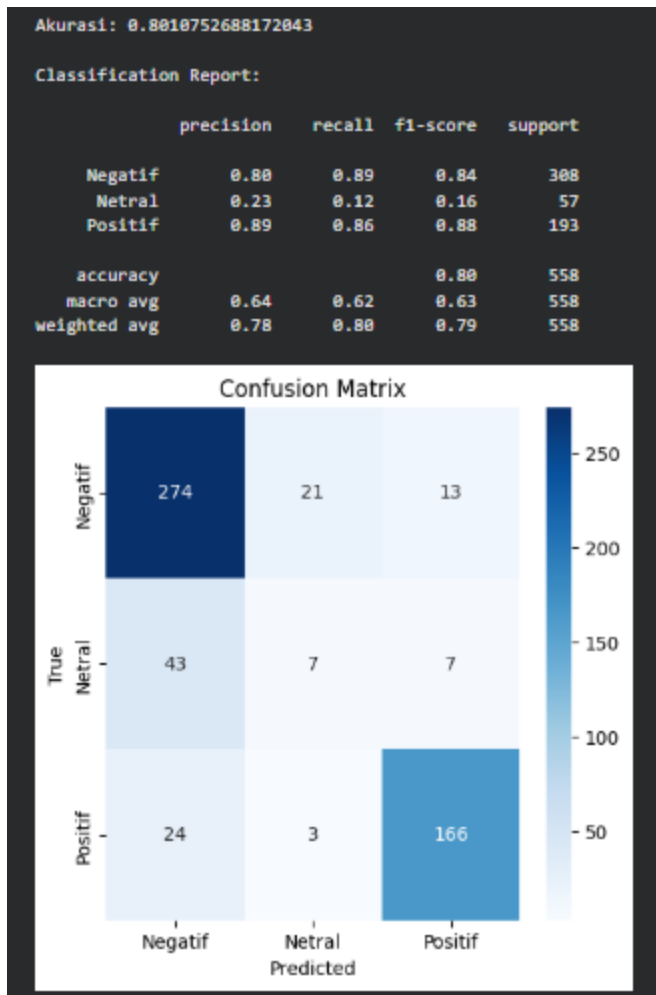
Berdasarkan hasil tuning menggunakan GridSearchCV, diperoleh model terbaik Support Vector Machine (LinearSVC) dengan parameter $C=1$ dan `loss=squared_hinge`. Model ini dilatih menggunakan data hasil oversampling SMOTE dan pengaturan `class_weight='balanced'` untuk menangani ketidakseimbangan kelas. Hasil validasi silang menunjukkan nilai F1-score weighted sebesar 94%, yang mengindikasikan performa model yang sangat baik pada data latih.

```

# Evaluasi
print("Akurasi:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, target_names=label_classes))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(5,5))
sns.heatmap(cm, annot=True, fmt='d',
            xticklabels=label_classes,
            yticklabels=label_classes,
            cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix")
plt.show()

```



Keterangan:

Untuk meningkatkan kinerja klasifikasi, dilakukan perbaikan dengan mengganti algoritma menjadi Support Vector Machine (SVM) serta menerapkan penyesuaian parameter dan penanganan ketidakseimbangan data menggunakan `class_weight = balanced`. Selain itu, proses ekstraksi fitur menggunakan TF-IDF terbukti mampu merepresentasikan data teks dengan lebih baik.

Hasil pengujian menunjukkan bahwa model SVM (LinearSVC) mampu meningkatkan performa klasifikasi dengan menghasilkan akurasi sebesar 80.82%, yang lebih tinggi dibandingkan dengan Decision Tree. Model SVM menunjukkan kinerja yang baik pada kelas negatif dan positif, dengan nilai F1-score masing-masing sebesar 84% dan 88%. Namun, performa pada kelas netral masih rendah, yang disebabkan oleh jumlah data netral yang relatif sedikit serta karakteristik teks yang tumpang tindih dengan kelas lainnya.

Dapat disimpulkan bahwa Support Vector Machine (SVM) lebih efektif digunakan untuk tugas analisis sentimen berbasis teks dibandingkan Decision Tree pada dataset ini. Meskipun

demikian, ketidakseimbangan data masih menjadi tantangan utama, sehingga pada penelitian selanjutnya disarankan untuk menggunakan teknik penanganan imbalance data yang lebih lanjut atau menambah jumlah data pada kelas minoritas agar performa model dapat meningkat secara lebih merata pada seluruh kelas.

Sumber Daya dan Referensi

1. Dataset: Kaggle, UCI Machine Learning Repository, Google Dataset Search
2. Referensi utama:
 - McKinney, W. (2017). *Python for Data Analysis*. O'Reilly Media.
 - Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn*. O'Reilly Media.

Selamat Belajar dan Berpraktikum!

“Data is the new oil, but Python is the refinery.”